



UNIVERSIDADE FEDERAL DE CAMPINA GRANDE – UFCG
CENTRO DE ENGENHARIA ELÉTRICA E INFORMÁTICA – CEEI
DEPARTAMENTO DE ENGENHARIA ELÉTRICA – DEE

Valmir Ferreira da Silva

**Implementação em Hardware de um Acelerador com
Arranjo Sistólico para Multiplicação de Matrizes de Ordem
N**

Campina Grande - PB

20 de maio de 2026

Valmir Ferreira da Silva

Implementação em Hardware de um Acelerador com Arranjo Sistólico para Multiplicação de Matrizes de Ordem N

Projeto de Engenharia Elétrica submetido à Coordenadoria de Graduação em Engenharia Elétrica da Universidade Federal de Campina Grande como parte dos requisitos necessários para a obtenção do grau de Bacharel em Engenharia Elétrica.

Orientador: Prof. Gutemberg Gonçalves dos Santos Júnior, D.Sc.

Campina Grande - PB

20 de maio de 2026

Valmir Ferreira da Silva

**Implementação em Hardware de um Acelerador com
Arranjo Sistólico para Multiplicação de Matrizes de Ordem
N**

**Prof. Gutemberg Gonçalves dos Santos
Júnior, D.Sc.**
Orientador

**Prof. Marcos Ricardo Alcantara
Morais, D.Sc.**
Convidado

Campina Grande - PB
20 de maio de 2026

A meus pais e meus irmãos, que sempre me apoiaram.

Agradecimentos

Minha jornada foi marcada por pessoas especiais que estiveram presentes em cada etapa, sempre prontas a me apoiar. Tenho plena consciência de que, sem elas, eu não estaria onde estou hoje. A todas, dedico os meus mais sinceros agradecimentos.

Aos meus pais e meus irmãos, pilares e porto seguro da minha vida: vocês acreditaram em mim desde o primeiro instante e sei que estarão comigo até o fim. Seus ensinamentos, carinho, apoio e acolhimento foram fundamentais para que eu pudesse seguir em frente e superar todas as dificuldades. Vocês me inspiram diariamente a ser uma pessoa melhor.

Aos meus amigos de Campina Grande: Arthur Henrique, Juliana Pessoa, Lucas Ventura, obrigado por compartilhar tantos momentos felizes dentro e fora da universidade. Vocês me ajudaram a perceber que os estudos não definem toda a minha vida, mas fazem parte dela. Com vocês, aprendi que é possível ir além dos portões da universidade, viver intensamente e desfrutar de todas as emoções que a vida proporciona.

Aos meus amigos do XMEN: Luiz, Joseilton, Davi, Maria, Cleisson, Túlio, Pedro Henrique, Pedro Arthur, Lucas, Agripino e José Ewerton, minha gratidão por se tornarem uma família escolhida. Obrigado por estarem sempre ao meu lado, me apoiando, animando e fazendo com que eu me sentisse acolhido. Vocês terão sempre um lugar especial no meu coração, não apenas pelos bolos, risadas e encontros, mas pelo companheirismo, amizade e conquistas que vivemos juntos. Tenho certeza de que ainda alcançaremos muito mais.

Aos professores Gutenberg, Marcos e Georgina, agradeço pela orientação, pelo conhecimento compartilhado e pelo apoio constante. Vocês me proporcionaram conquistas valiosas dentro e fora da academia. Mais do que mestres, foram companheiros e amigos, dividindo risadas, histórias e vitórias. A inspiração que vocês me deram certamente também alcança muitas outras pessoas.

Resumo

Este trabalho apresenta o projeto, a implementação e a validação experimental de um acelerador em hardware para multiplicação de matrizes baseado em arranjos sistólicos, desenvolvido em *SystemVerilog* e implementado na plataforma *FPGA Zynq UltraScale+ MPSoC ZCU106*. A arquitetura proposta foi concebida de forma parametrizável, permitindo a avaliação de diferentes dimensões do arranjo sistólico quanto a desempenho, latência, utilização de recursos, consumo de potência e escalabilidade. O desenvolvimento envolveu a modelagem do fluxo de dados, a implementação dos elementos de processamento, dos subsistemas de memória e da unidade de controle, bem como a integração com mecanismos de comunicação entre o *Processing System* e a *Programmable Logic* por meio de AXI DMA. A verificação funcional foi realizada em nível RTL, com resultados consistentes em relação ao modelo de referência em software. Os resultados experimentais demonstram que a arquitetura proposta é capaz de acelerar significativamente a multiplicação de matrizes, alcançando ganhos expressivos de desempenho para matrizes maiores, embora com aumento no consumo de recursos lógicos e maior dificuldade de fechamento de temporização em arranjos de grande dimensão. Os resultados também evidenciam que o principal gargalo do sistema completo está associado à comunicação de dados entre software e hardware, e não ao núcleo computacional do arranjo sistólico. Dessa forma, o trabalho confirma o potencial do uso de arquiteturas sistólicas em FPGA como solução eficiente para aplicações intensivas em processamento matricial.

Palavras-chave: arranjo sistólico; multiplicação de matrizes; FPGA; Zynq UltraScale+ MPSoC; acelerador em hardware; SystemVerilog; DMA; computação paralela.

Abstract

This work presents the design, implementation, and experimental validation of a hardware accelerator for matrix multiplication based on systolic arrays, developed in *SystemVerilog* and implemented on the *Zynq UltraScale+ MPSoC ZCU106 FPGA* platform. The proposed architecture was designed in a parameterizable manner, allowing the evaluation of different systolic array dimensions in terms of performance, latency, resource utilization, power consumption, and scalability. The development included dataflow modeling, implementation of processing elements, memory subsystems, and control logic, as well as integration with communication mechanisms between the *Processing System* and the *Programmable Logic* through AXI DMA. Functional verification was carried out at the RTL level, with results consistent with the software reference model. Experimental results show that the proposed architecture can significantly accelerate matrix multiplication, achieving expressive performance gains for larger matrices, although with increased logical resource usage and greater timing closure challenges for large array sizes. The results also indicate that the main bottleneck of the complete system is associated with data communication between software and hardware rather than with the computational core of the systolic array. Therefore, this work confirms the potential of FPGA-based systolic architectures as an efficient solution for matrix-intensive computing applications.

Keywords: systolic array; matrix multiplication; FPGA; Zynq UltraScale+ MPSoC; hardware accelerator; SystemVerilog; DMA; parallel computing.

Lista de ilustrações

Figura 1 – Algoritmo clássico de multiplicação de matrizes	16
Figura 2 – Estrutura conceitual de um arranjo sistólico bidimensional composto por elementos de processamento localmente conectados.	18
Figura 3 – Fluxo de dados em um arranjo sistólico 3×3 , destacando a propagação dos operandos ao longo dos PEs e o cálculo paralelo dos elementos da matriz resultante.	19
Figura 4 – Fluxo de dados no arranjo sistólico 3×3 em uma janela amostral correspondente aos ciclos 1, 2 e 3 de clock.	20
Figura 5 – Fluxo de dados no arranjo sistólico 3×3 em uma janela amostral correspondente aos ciclos 4, 5 e 6 de clock.	20
Figura 6 – Fluxo de dados no arranjo sistólico 3×3 nos ciclos finais de processamento, seguido da matriz resultante da operação.	21
Figura 7 – Diagrama de Blocos do Sistema	27
Figura 8 – Diagrama de Blocos do Zynq UltraScale+ MPSoC	27
Figura 9 – Visão funcional de um elemento de processamento do arranjo sistólico.	29
Figura 10 – Estrutura interna de um elemento de processamento.	30
Figura 11 – Diagrama representativo do MAC.	31
Figura 12 – Comunicação local e propagação de dados entre elementos de processamento em um arranjo sistólico.	32
Figura 13 – Diagrama do Deslocador de Matrizes	33
Figura 14 – Exemplo do Escalonador de Matrizes	34
Figura 15 – Simulação do Escalonador de Matrizes 32x32-4 bits por símbolo	35
Figura 16 – Bloco do Subsistema de Entrada	35
Figura 17 – Máquina de Estados do Subsistema de Memória de Entrada	36
Figura 18 – Máquina de Estados do Subsistema de Cálculo Matricial	37
Figura 19 – Bloco do Subsistema da Unidade de Arranjo Sistólico	38
Figura 20 – Bloco do Subsistema de Saída	39
Figura 21 – Máquina de Estados do Subsistema de Memória de Saída	41
Figura 22 – Máquina de Estados da Unidade de Controle Geral	42
Figura 23 – Diagrama de Blocos do Ambiente de Verificação do Acelerador Sistólico	45
Figura 24 – Simulação de uma Multiplicação $I \times B = B$, 32×32 - 4 Bits por Símbolo.	45
Figura 25 – Slack de setup e hold em função do tamanho do arranjo sistólico a @333 MHz.	47
Figura 26 – Comparação entre throughput teórico (matrizes/s) e throughput medido (matrizes/s) para diferentes tamanhos de arranjos sistólicos.	48

Figura 27 – LUTs, registradores e CLBs em função do tamanho do arranjo sistólico a 333 MHz.	50
Figura 28 – Estimativa de Potência vs Tamanho do Arranjo Sistólico @333 MHz . .	51
Figura 29 – Decomposição de potencia dinâmica para o tamanho de arranjo sistólico de 64x64	52
Figura 30 – Comparação entre execução em software e desempenho do acelerador FPGA.	53
Figura 31 – Comparação entre o acelerador Ps e a execução teórica do arranjo sistólico. .	54
Figura 32 – Speedup do acelerador FPGA em relação à implementação em software para diferentes tamanhos do arranjo sistólico.	55

Lista de tabelas

Tabela 1 – Resumo das versões estáveis do acelerador desenvolvidas ao longo do projeto	26
Tabela 2 – Evolução temporal do escalonador de matrizes para A e B	34
Tabela 3 – Latência observada nas simulações em ciclos de clock para cada etapa do acelerador nas diferentes versões da arquitetura	46
Tabela 4 – Evolução da arquitetura do acelerador	46
Tabela 5 – Utilização de recursos do FPGA para diferentes tamanhos do arranjo sistólico.	49

Lista de abreviaturas e siglas

CNN	Convolutional Neural Network
DNN	Deep Neural Network
FPGA	Field-Programmable Gate Array
RTL	Register Transfer Level
VLSI	Very-Large-Scale Integration
HIL	Hardware-in-the-Loop
AMD	Advanced Micro Devices
MPSoC	Multi-Processor System-on-Chip
CPU	Central Processing Unit
BIT	Binary Digit
DUT	Design Under Test
PS	Processing System
PL	Programmable Logic
SAIF	Switching Activity Interchange Format
STA	Static Timing Analysis
LUTs	Look-Up Tables
CLBs	Configurable Logic Blocks
ISA	Instruction Set Architecture

Sumário

1	INTRODUÇÃO	14
1.1	Objetivos	15
1.1.1	Objetivo Geral	15
1.1.2	Objetivos Específicos	15
2	FUNDAMENTAÇÃO TEÓRICA	16
2.1	Multiplicação de Matrizes	16
2.2	Arranjos Sistólicos	17
2.2.1	Conceito de Arquitetura Sistólica	17
2.2.2	Fluxo de Dados no Arranjo	18
2.3	FPGA (Field-Programmable Gate Array)	21
2.3.1	Arquitetura Interna	22
2.3.2	Blocos Lógicos, DSP e Memória	22
2.3.3	Vantagens para Aceleração de Hardware	23
2.4	Linguagens de Descrição de Hardware	24
2.4.1	Projeto em Nível RTL	24
2.4.2	Verilog/SystemVerilog	24
3	PROJETO E IMPLEMENTAÇÃO DO HARDWARE	25
3.1	Considerações Iniciais	25
3.2	Diagrama em Blocos	26
3.3	Fluxo de Dados do Acelerador	28
3.3.1	Parametrização do Arranjo Sistólico	28
3.3.2	Elemento de Processamento (Processing Element – PE)	29
3.3.3	Estrutura Interna do PE	29
3.3.4	Unidade Multiplicadora-Acumuladora (MAC)	30
3.3.5	Comunicação e Propagação de Dados entre PEs	31
3.3.6	Escalonamento das Entradas	32
3.4	Subsistemas e Controle Geral	35
3.4.1	Subsistema de Memória de Entrada	35
3.4.2	Unidade de Arranjo Sistólico	37
3.4.3	Subsistema de Memória de Saída	39
3.4.4	Unidade de Controle e Sequenciamento Operacional	41
4	SIMULAÇÃO E ANÁLISES	44
4.1	Verificação Funcional	44

4.1.1	Metodologia de Teste	44
4.1.2	Resultados de Simulação	44
4.1.3	Latência	46
4.1.4	Análise de Temporização (Timing)	46
4.1.5	Throughput Teórico x Mensurado	47
4.1.6	Utilização do FPGA	49
4.1.7	Potência Consumida	50
4.2	Comparação com Implementação em Software	53
4.2.1	Implementação em CPU	53
4.2.2	Comparação com o Acelerador em Hardware	53
4.2.3	Análise de Speedup	54
5	CONCLUSÕES E TRABALHOS FUTUROS	56
5.1	Síntese dos Resultados Obtidos	56
5.2	Limitações do Trabalho	57
5.3	Possíveis Melhorias	57
5.4	Direções para Trabalhos Futuros	58
	REFERÊNCIAS	60

1 Introdução

A multiplicação de matrizes constitui uma operação fundamental em diversas áreas do conhecimento, como matemática, estatística, ciência da computação e engenharia, estando presente em inúmeros algoritmos científicos e aplicações industriais. Com o avanço recente da visão computacional e dos modelos de aprendizado profundo, em especial as Convolutional Neural Networks (*CNNs*) e as Deep Neural Networks (*DNNs*), a demanda por operações de multiplicação de matrizes cresceu significativamente, tanto em volume quanto em complexidade computacional.

Nos modelos baseados em aprendizado profundo, grande parte do custo computacional está associada às operações convolucionais e às camadas totalmente conectadas, as quais podem ser reorganizadas e executadas na forma de multiplicações de matrizes densas. Como consequência, essas operações passaram a representar um dos principais limitantes de desempenho e eficiência energética desses sistemas. Esse cenário evidencia as limitações de abordagens puramente baseadas em software, tornando-as insuficientes para atender às exigências de desempenho dessas aplicações e motivando o desenvolvimento de arquiteturas de hardware dedicadas capazes de explorar paralelismo massivo e maior eficiência de processamento.

Entre essas arquiteturas, destacam-se os arranjos sistólicos (*systolic arrays*), introduzidos por Kung e Leiserson no contexto de sistemas *VLSI*. Essas arquiteturas exploram o paralelismo espacial e temporal aliado a um fluxo regular de dados, permitindo elevadas taxas de processamento e eficiência energética na execução de operações matriciais, características particularmente atrativas para aceleradores de computação intensiva (KUNG; LEISERSON, 1979).

Nesse contexto, o presente trabalho propõe projetar, implementar e validar um acelerador de multiplicação de matrizes baseado em arranjos sistólicos (*systolic arrays*), implementado em uma *FPGA Zynq™ UltraScale+™ MPSoC ZCU106 Evaluation Kit*. O desenvolvimento envolve o estudo de diferentes tamanhos de arranjos sistólicos, a modelagem detalhada do fluxo de dados e da arquitetura em nível *RTL*, o desenvolvimento de ambiente de verificação e testes *Hardware-in-the-Loop*, bem como a avaliação experimental de desempenho, consumo de recursos, escalabilidade e comparação com soluções reportadas na literatura.

1.1 Objetivos

1.1.1 Objetivo Geral

Projetar, implementar e validar um acelerador de multiplicação de matrizes baseado em arranjos sistólicos (*systolic arrays*), utilizando a *FPGA Zynq™ UltraScale+™ MPSoC ZCU106 Evaluation Kit*, com o propósito de avaliar desempenho, utilização de recursos, eficiência computacional e escalabilidade da arquitetura proposta.

1.1.2 Objetivos Específicos

- Modelar o fluxo de dados da multiplicação de matrizes e descrever o comportamento da arquitetura proposta, definindo protocolos, caminhos de dados e mecanismos de sincronização.
- Projetar os módulos auxiliares do acelerador, incluindo unidade de controle, buffers de entrada e saída, interconexões internas, interfaces de comunicação e lógica de sincronização do fluxo de dados do arranjo sistólico.
- Realizar verificação funcional em nível *RTL*, empregando metodologias modernas de testbench, garantindo cobertura adequada e conformidade com a especificação comportamental.
- Conduzir as simulações e implementações em ambiente híbrido utilizando ferramentas *Cadence* e *AMD/Xilinx*, especificamente *Xcelium*, *SimVision*, *Vivado* e *VSCode* como principais plataformas de desenvolvimento.
- Desenvolver um ambiente em *Python* para *Hardware-in-the-Loop (HiL)*, conectando o sistema ao desktop e permitindo interação direta entre hardware e software, com suporte a testes em tempo real, depuração e validação da arquitetura.
- Efetuar o desenho hierárquico do projeto e realizar as etapas de implementação, verificação, integração e validação final do sistema seguindo boas práticas de desenvolvimento de hardware digital.
- Avaliar a escalabilidade do acelerador para diferentes ordens de matrizes, verificando os impactos no desempenho e na utilização de recursos da *FPGA*.
- Comparar o desempenho obtido com resultados de trabalhos relevantes da literatura e identificar pontos fortes, limitações e potenciais melhorias da arquitetura proposta.

2 Fundamentação teórica

2.1 Multiplicação de Matrizes

Considere duas matrizes $A \in R^{M \times K}$ e $B \in R^{K \times N}$, o produto matricial $C = A \times B$ resulta em uma matriz $C \in R^{M \times N}$, cujos elementos são definidos:

$$A = \begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0k} \\ a_{10} & a_{11} & \cdots & a_{1k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m0} & a_{m1} & \cdots & a_{mk} \end{bmatrix} \quad \text{e} \quad B = \begin{bmatrix} b_{00} & b_{01} & \cdots & b_{0n} \\ b_{10} & b_{11} & \cdots & b_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{k0} & b_{k1} & \cdots & b_{kn} \end{bmatrix} \quad (2.1)$$

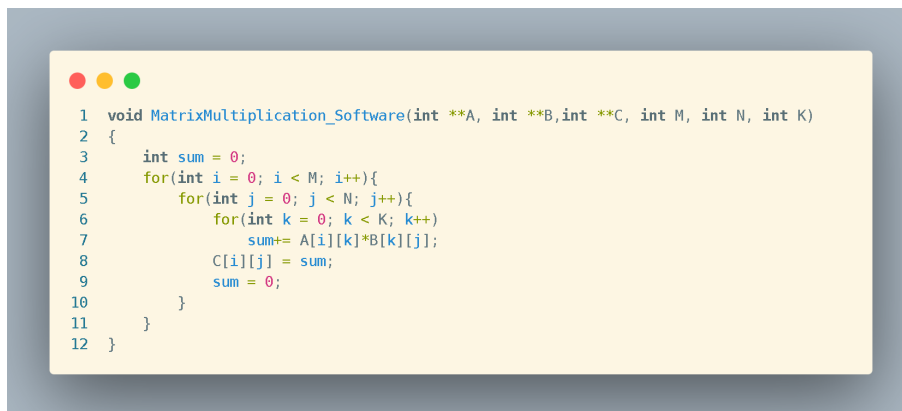
$$c_{ij} = \sum_{k=0}^K a_{ik} \times b_{kj} \quad (2.2)$$

A matriz resultante será:

$$C = \begin{bmatrix} c_{00} & c_{01} & \cdots & c_{0n} \\ c_{10} & c_{11} & \cdots & c_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m0} & c_{m1} & \cdots & c_{mn} \end{bmatrix} \quad (2.3)$$

O algoritmo clássico para multiplicar duas matrizes A e B é ilustrado a seguir.

Figura 1 – Algoritmo clássico de multiplicação de matrizes



```

1 void MatrixMultiplication_Software(int **A, int **B, int **C, int M, int N, int K)
2 {
3     int sum = 0;
4     for(int i = 0; i < M; i++){
5         for(int j = 0; j < N; j++){
6             for(int k = 0; k < K; k++){
7                 sum += A[i][k]*B[k][j];
8                 C[i][j] = sum;
9                 sum = 0;
10            }
11        }
12    }

```

Autoria: Própria.

Por exemplo, para o caso especial em que A e $B \in R^{K \times K}$ a complexidade dessa operação será $\mathcal{O}(K^3)$. Considerando o caso geral, temos que a operação representa complexidade computacional assintótica de $\mathcal{O}(M \times N \times K)$. Ao implementarmos este algoritmo

em arquiteturas convencionais baseadas em processadores de propósito geral (CPU), o algoritmo é executado de forma sequencial e ineficiente, tornando-se custoso computacionalmente para matrizes de grandes dimensões (SMEDING, 2018; CHEN, 2018).

Em aplicações modernas, como aprendizado profundo e visão computacional, a multiplicação de matrizes constitui o núcleo computacional de operações como convoluções e camadas totalmente conectadas, sendo responsável por parcela significativa do tempo de execução e do consumo energético dos sistemas.

2.2 Arranjos Sistólicos

2.2.1 Conceito de Arquitetura Sistólica

Os arranjos sistólicos constituem uma classe de arquiteturas paralelas especializadas projetadas para acelerar operações computacionalmente intensivas por meio da exploração de paralelismo espacial e temporal. Essas arquiteturas são compostas por redes regulares de unidades de processamento interconectadas denominadas *Processing Elements* (PEs), que operam de forma síncrona e trocam dados apenas com vizinhos locais.

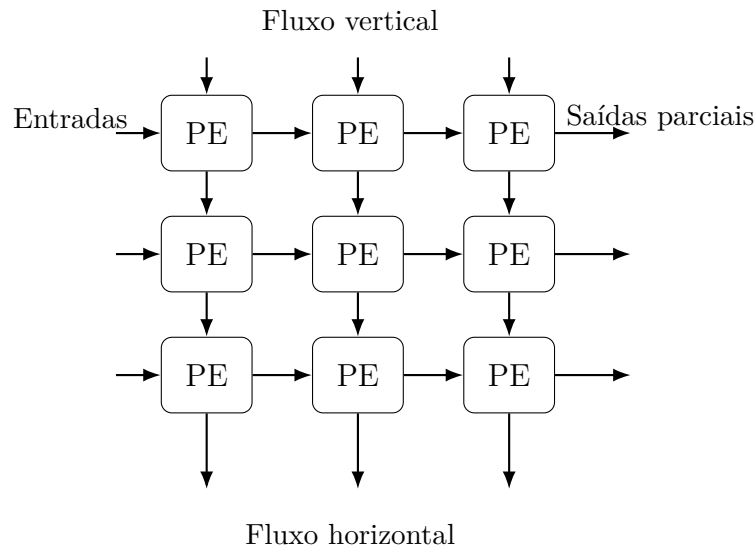
O termo “sistólico” deriva da analogia com o funcionamento do sistema circulatório humano, no qual o fluxo de dados percorre a rede de forma rítmica e sincronizada pelo relógio do sistema. Cada elemento de processamento realiza uma pequena parte da computação e transmite os resultados para os elementos adjacentes, formando um pipeline espacial distribuído.

A principal vantagem dessa organização é a reutilização eficiente de dados. Uma vez que um elemento de dados é carregado na matriz sistólica, ele pode ser reutilizado por diversos elementos de processamento à medida que se propaga pelo arranjo, reduzindo significativamente a necessidade de acessos à memória externa, que tradicionalmente constitui o principal gargalo em arquiteturas baseadas no modelo de Von Neumann (SYSTOLIC..., 2023).

Outra característica importante dessas arquiteturas é sua estrutura altamente regular, permitindo escalabilidade por replicação dos elementos de processamento. Isso facilita sua implementação em dispositivos reconfiguráveis, como FPGAs, nos quais a replicação de estruturas idênticas pode ser realizada de forma eficiente durante o processo de síntese e implementação física.

Conforme ilustrado na Figura 2, os dados fluem através da rede de elementos de processamento enquanto as operações aritméticas são executadas em pipeline, o que permite atingir elevadas taxas de processamento. Cabe ressaltar, ainda, que esse tipo de organização pode ser interpretado como não estacionária, uma vez que ambos os operandos são propagados simultaneamente pelo arranjo (KUNG; LEISERSON, 1979).

Figura 2 – Estrutura conceitual de um arranjo sistólico bidimensional composto por elementos de processamento localmente conectados.



2.2.2 Fluxo de Dados no Arranjo

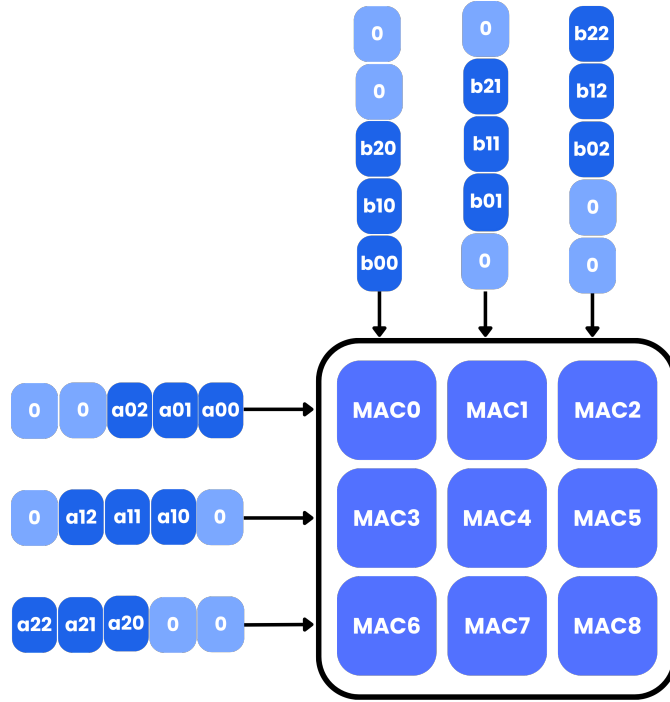
O fluxo de dados em um arranjo sistólico bidimensional é caracterizado pela propagação sincronizada dos operandos entre elementos de processamento (PEs), explorando paralelismo espacial e reutilização local de dados. Conforme ilustrado na Figura 3, na multiplicação $C = A \times B$, os elementos da matriz A são inseridos horizontalmente, enquanto os elementos de B são inseridos verticalmente, encontrando-se nos PEs, onde são realizadas operações de multiplicação e acumulação (MAC).

Para um arranjo 3×3 , os dados são introduzidos de forma escalonada ao longo dos ciclos de clock, incluindo uma fase inicial de preenchimento do pipeline. Durante essa fase, são inseridos valores nulos para alinhar temporalmente os operandos. À medida que os dados percorrem o arranjo, cada PE acumula resultados parciais correspondentes aos elementos da matriz C , permitindo a execução paralela das operações.

Após o preenchimento do pipeline, o arranjo entra em regime permanente, no qual novos resultados são produzidos a cada ciclo de clock. Como evidenciado na Figura 3, esse fluxo estruturado permite que múltiplos elementos da matriz resultante sejam computados simultaneamente em diferentes estágios do arranjo.

Esse modelo de comunicação local entre PEs reduz significativamente a necessidade de acesso à memória externa, aumentando o throughput e a eficiência energética do sistema. Essa característica é fundamental em aceleradores sistólicos, uma vez que o fluxo de dados previsível minimiza a sobrecarga de controle e otimiza o desempenho computacional (??).

Figura 3 – Fluxo de dados em um arranjo sistólico 3×3 , destacando a propagação dos operandos ao longo dos PEs e o cálculo paralelo dos elementos da matriz resultante.



Autoria: Própria.

Para tornar os conceitos apresentados mais concretos, realiza-se a seguir um estudo de caso em escala simplificada da multiplicação entre duas matrizes quadradas com o uso de um arranjo sistólico. Considere as matrizes A e B , apresentadas na equação 2.5, bem como a matriz B^T .

As Figuras 4, 5 e 6 ilustram, de forma sequencial, o fluxo de dados no interior de um arranjo sistólico 3×3 , evidenciando tanto a propagação dos operandos entre os PEs quanto a execução das operações de multiplicação e acumulação ao longo dos ciclos de clock.

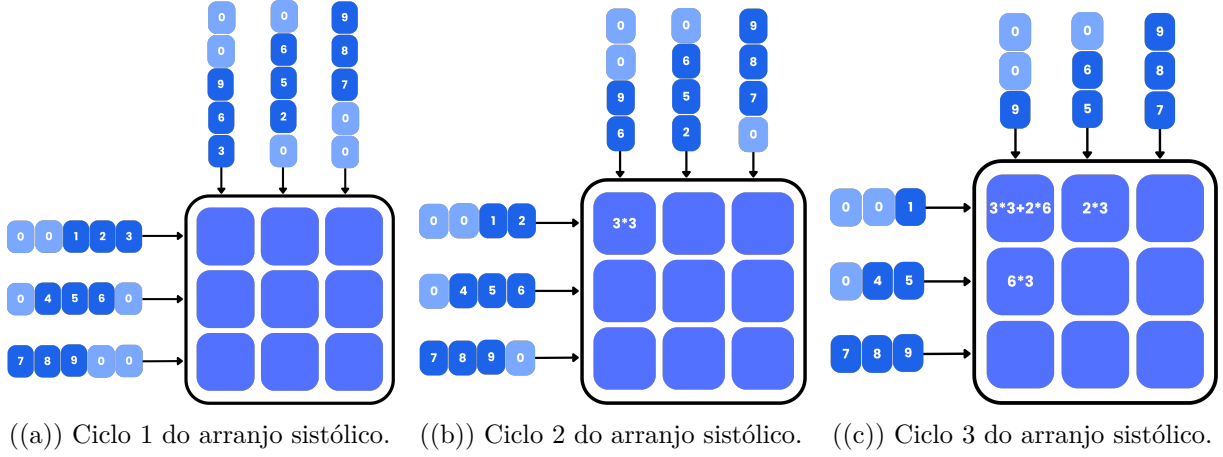
Com base nesse estudo de caso, é possível notar ainda que a latência total de cálculo da unidade de multiplicação de matrizes está diretamente relacionada ao tempo de propagação dos operandos ao longo do arranjo sistólico e ao tempo necessário para que os resultados sejam completamente disponibilizados na saída. Para um arranjo quadrado de dimensão $N \times N$, essa latência total pode ser modelada por

$$L_{\text{arranjo}} = (2N - 1) + N \quad (2.4)$$

em que $(2N - 1)$ representa o atraso associado ao preenchimento e ao deslocamento dos dados no arranjo, e N corresponde ao tempo adicional de escoamento necessário para a saída completa dos resultados.

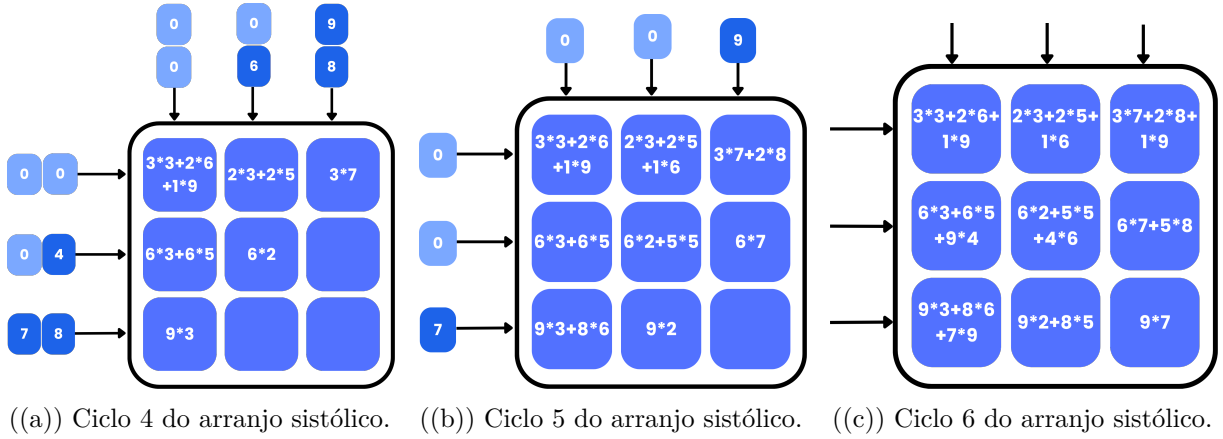
$$A = \begin{bmatrix} 3 & 2 & 1 \\ 6 & 5 & 4 \\ 9 & 8 & 7 \end{bmatrix}, \quad B = \begin{bmatrix} 3 & 2 & 7 \\ 6 & 5 & 8 \\ 9 & 6 & 9 \end{bmatrix} \quad \text{e} \quad B^T = \begin{bmatrix} 3 & 6 & 9 \\ 2 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \quad (2.5)$$

Figura 4 – Fluxo de dados no arranjo sistólico 3×3 em uma janela amostral correspondente aos ciclos 1, 2 e 3 de clock.



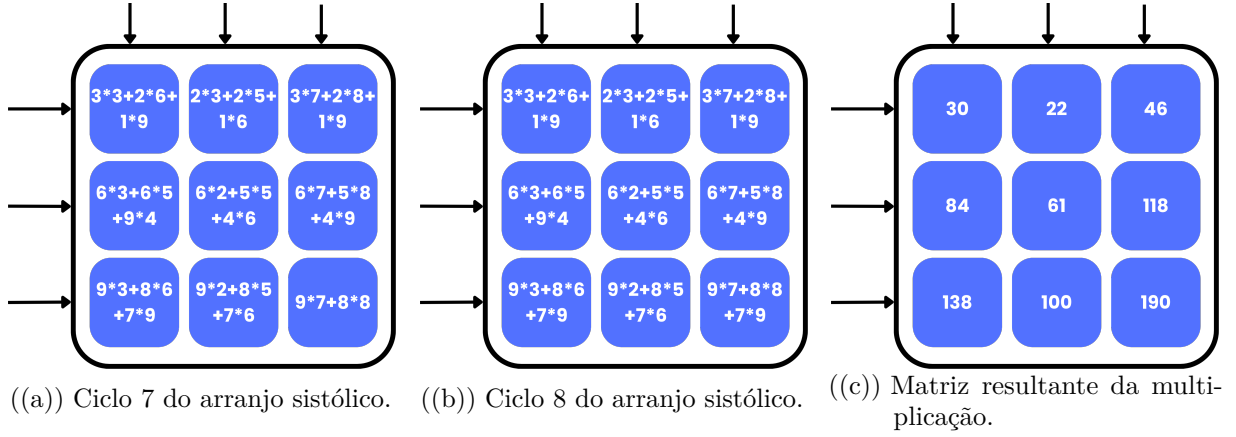
Autoria: Própria.

Figura 5 – Fluxo de dados no arranjo sistólico 3×3 em uma janela amostral correspondente aos ciclos 4, 5 e 6 de clock.



Autoria: Própria.

Figura 6 – Fluxo de dados no arranjo sistólico 3×3 nos ciclos finais de processamento, seguido da matriz resultante da operação.



Autoria: Própria.

2.3 FPGA (Field-Programmable Gate Array)

Os *Field-Programmable Gate Arrays* (FPGAs) são dispositivos semicondutores reconfiguráveis que permitem a implementação de circuitos digitais personalizados após o processo de fabricação. Diferentemente de circuitos integrados de aplicação específica (ASICs), os FPGAs podem ser programados repetidamente, permitindo a exploração de diferentes arquiteturas e otimizações de hardware sem necessidade de novas etapas de fabricação.

Entre as plataformas modernas baseadas em FPGA destaca-se o dispositivo *AMD Zynq UltraScale+ MPSoC*, presente no kit de avaliação ZCU106. De acordo com o manual da placa, a ZCU106 é uma plataforma de prototipagem de propósito geral baseada no dispositivo *XCZU7EV-2FFVC1156*, integrando, em um único chip, um sistema de processamento com quatro núcleos Arm Cortex-A53, dois núcleos Arm Cortex-R5F e lógica programável para implementação de hardware dedicado (XILINX, INC., 2019).

O Zynq UltraScale+ MPSoC possui dois domínios principais: o *Processing System* (PS) e o *Programmable Logic* (PL). Conforme descrito no manual da ZCU106, o PS reúne os processadores de aplicação e de tempo real, enquanto o PL corresponde à malha reconfigurável utilizada para implementação de aceleradores e circuitos digitais personalizados (XILINX, INC., 2019).

No kit de avaliação ZCU106, o dispositivo disponibiliza recursos relevantes para implementação de arquiteturas paralelas, incluindo aproximadamente 460,8 mil *CLB flip-flops*, 1728 *DSP slices*, 11 Mb de *Block RAM* e 27 Mb de *UltraRAM*, além de 504 mil *logic cells*, conforme resumido na Tabela 1-1 do manual da placa (XILINX, INC., 2019). Esses valores servem como referência para a análise de ocupação de recursos e

escalabilidade dos aceleradores implementados neste trabalho. A comunicação entre o PS e o PL ocorre por meio de interfaces internas de alta largura de banda, permitindo a integração eficiente entre software e hardware dedicado. Segundo o manual da ZCU106, essa organização torna a plataforma adequada para prototipagem rápida e desenvolvimento de sistemas heterogêneos com aceleradores em FPGA (XILINX, INC., 2019).

2.3.1 Arquitetura Interna

A arquitetura interna de um FPGA é composta por uma rede regular de blocos lógicos programáveis interconectados por uma malha de roteamento configurável. Essa estrutura permite implementar desde circuitos combinacionais simples até sistemas digitais complexos.

Nos dispositivos da família UltraScale+, a lógica programável é organizada em *Configurable Logic Blocks* (CLBs), responsáveis pela implementação das funções lógicas e do armazenamento temporário de dados. Cada CLB contém múltiplas *Look-Up Tables* (LUTs), que implementam funções lógicas arbitrárias, além de registradores utilizados para armazenamento síncrono.

No kit de avaliação ZCU106, a região de lógica programável do dispositivo disponibiliza aproximadamente 230 400 LUTs, 460 800 registradores e 28 800 CLBs, recursos que podem ser utilizados para a implementação de arquiteturas digitais paralelas e aceleradores de hardware. Esses valores representam a capacidade total do dispositivo e servem como referência para avaliar a utilização de recursos nos projetos implementados.

A interconexão entre os blocos lógicos é realizada por meio de uma rede de roteamento programável composta por multiplexadores e trilhas metálicas distribuídas pelo dispositivo. Durante o processo de implementação, as ferramentas de síntese e *place-and-route* determinam como cada elemento lógico será posicionado no dispositivo e como as conexões serão realizadas.

Além da lógica programável, os FPGAs modernos incluem diversos blocos especializados capazes de executar operações específicas de forma mais eficiente, tais como multiplicações, operações aritméticas e armazenamento de dados.

2.3.2 Blocos Lógicos, DSP e Memória

Os dispositivos UltraScale+ incorporam diferentes tipos de recursos de hardware dedicados que complementam os CLBs e permitem a implementação eficiente de sistemas digitais complexos.

Os *Configurable Logic Blocks* (CLBs) são os elementos fundamentais da lógica programável e são compostos principalmente por LUTs e registradores. As LUTs permi-

tem implementar funções lógicas combinacionais enquanto os registradores permitem a construção de circuitos sequenciais e pipelines.

Além dos CLBs, os dispositivos incluem blocos de processamento digital dedicados denominados *DSP Slices*. Esses blocos são otimizados para operações aritméticas de alta performance, como multiplicações, acumulações e operações MAC (*Multiply-Accumulate*), sendo amplamente utilizados em aplicações de processamento de sinais e aprendizado de máquina.

Outro recurso importante são as memórias internas do tipo *Block RAM* (BRAM) e *UltraRAM*. Esses blocos permitem a implementação de buffers, caches e estruturas de armazenamento temporário diretamente dentro do FPGA, reduzindo a necessidade de acesso à memória externa e aumentando significativamente o desempenho do sistema.

No contexto de aceleradores de hardware, esses recursos podem ser utilizados para implementar unidades de processamento altamente paralelas, armazenar dados intermediários e reduzir gargalos de comunicação com o processador hospedeiro.

2.3.3 Vantagens para Aceleração de Hardware

Uma das principais vantagens do uso de FPGAs em sistemas computacionais é a possibilidade de implementar arquiteturas altamente paralelas e especializadas para determinadas aplicações.

Enquanto processadores convencionais executam instruções sequencialmente, aceleradores implementados em FPGA podem explorar paralelismo em nível de dados e em nível de pipeline. Isso permite que múltiplas operações sejam realizadas simultaneamente, aumentando significativamente o desempenho em aplicações intensivas em cálculo.

Outra vantagem importante é a proximidade entre os recursos computacionais e as memórias internas do dispositivo. Ao armazenar dados intermediários diretamente em BRAM ou UltraRAM, é possível reduzir a latência de acesso à memória e minimizar a transferência de dados para memórias externas.

Além disso, a integração entre o processador ARM e a lógica programável presente no Zynq UltraScale+ MPSoC permite a construção de sistemas heterogêneos eficientes, nos quais o software executado no PS controla aceleradores de hardware implementados no PL.

2.4 Linguagens de Descrição de Hardware

2.4.1 Projeto em Nível RTL

O nível de abstração mais comum utilizado no desenvolvimento de circuitos digitais é o nível *Register Transfer Level* (RTL). Nesse nível, o comportamento do circuito é descrito em termos de transferências de dados entre registradores e das operações lógicas realizadas sobre esses dados.

No modelo RTL, o projetista especifica explicitamente os registradores, os caminhos de dados e a lógica de controle do sistema. Essa abordagem permite que ferramentas de síntese transformem a descrição em hardware físico mantendo a correspondência funcional com o modelo descrito.

Projetos desenvolvidos em RTL podem ser simulados antes da síntese, permitindo verificar a corretude funcional do sistema por meio de *testbenches*. Após essa etapa, a descrição é processada pelas ferramentas de síntese e implementação que geram a configuração final do FPGA.

2.4.2 Verilog/SystemVerilog

O Verilog é uma das linguagens de descrição de hardware mais utilizadas para o desenvolvimento de circuitos digitais e sistemas em FPGA. Sua sintaxe é inspirada em linguagens de programação como C, facilitando a adoção por engenheiros de computação e eletrônica.

O SystemVerilog é uma extensão do Verilog que incorpora recursos adicionais para modelagem de hardware, verificação funcional e descrição de sistemas complexos. Entre suas principais melhorias estão novos tipos de dados, estruturas de controle mais robustas e suporte avançado para verificação baseada em objetos.

No desenvolvimento de aceleradores de hardware, o SystemVerilog permite descrever arquiteturas parametrizáveis e altamente paralelas, facilitando a criação de módulos reutilizáveis e escaláveis. Esses recursos são particularmente úteis na implementação de estruturas como arranjos sistólicos, pipelines e unidades de processamento especializadas.

Assim, a utilização de SystemVerilog em conjunto com ferramentas de síntese e implementação permite transformar descrições em nível RTL em circuitos digitais eficientes executados diretamente no hardware programável do FPGA.

3 Projeto e Implementação do Hardware

3.1 Considerações Iniciais

Ao longo do desenvolvimento do sistema foram implementadas quatro versões estáveis do acelerador, cada uma delas representando uma evolução em termos de infraestrutura de comunicação, organização arquitetural e eficiência no fluxo de dados.

A primeira versão foi implementada utilizando a plataforma *Arty A7*, na qual a comunicação entre o sistema hospedeiro e o acelerador era realizada por meio de uma interface UART. Essa implementação teve como principal objetivo validar o funcionamento da arquitetura do arranjo sistólico e da unidade de controle. Entretanto, verificou-se que a baixa largura de banda da interface serial representava um gargalo significativo para o sistema. Além disso, os recursos disponíveis na FPGA limitaram a escalabilidade da arquitetura, restringindo os experimentos a matrizes de pequenas dimensões, tipicamente inferiores a 16×16 .

Para superar essas limitações, a segunda versão do sistema adotou a plataforma *Zynq ZCU106*, baseada em um *System-on-Chip* que integra um processador ARM ao subsistema de lógica programável. Essa mudança permitiu uma reestruturação significativa da infraestrutura de comunicação. A interface UART foi substituída por uma comunicação baseada em Ethernet, enquanto o *Processing System* passou a executar um servidor implementado sobre o sistema operacional em tempo real *FreeRTOS*. No nível de comunicação entre o *Processing System* (PS) e a *Programmable Logic* (PL), foi adotada uma interface baseada em *AXI Stream FIFO*, permitindo maior largura de banda na transferência de dados.

Durante os testes dessa segunda versão foi identificado um novo gargalo relacionado à preparação dos dados de entrada do acelerador. Para atender aos requisitos de sincronização do arranjo sistólico, as matrizes precisavam ser previamente deslocadas (*skewed*) antes de serem transmitidas ao acelerador. Esse processo aumentava significativamente o volume de dados transferidos e introduzia latência adicional na comunicação entre PS e PL. Nesse modelo, cada matriz exigia aproximadamente $N(2N - 1)$ transferências quando os elementos eram enviados individualmente.

Na terceira versão do sistema foi realizada uma modificação arquitetural no acelerador para eliminar essa limitação. O módulo responsável pelo deslocamento das matrizes foi incorporado diretamente ao hardware, permitindo que os dados fossem enviados ao acelerador em seu formato convencional. Com essa alteração, o custo de transferência foi reduzido para N^2 , simplificando o fluxo de dados e reduzindo a sobrecarga de comunicação.

Por fim, na quarta versão foi introduzida uma otimização adicional na infraestrutura de comunicação entre PS e PL por meio da integração de um mecanismo de *Direct Memory Access* (DMA). Esse mecanismo permite a transferência direta de blocos de memória entre o sistema e o acelerador, sem a intervenção direta do processador. Além disso, os dados passaram a ser transmitidos em blocos correspondentes a linhas completas das matrizes, o que reduziu significativamente a latência de comunicação e aumentou a eficiência do sistema.

Essa evolução arquitetural resultou em melhorias significativas no desempenho do acelerador. Nos capítulos seguintes são apresentados os detalhes da arquitetura do sistema e os resultados experimentais obtidos com a versão final do acelerador. A Tabela 1 mostra a evolução do sistema.

Tabela 1 – Resumo das versões estáveis do acelerador desenvolvidas ao longo do projeto

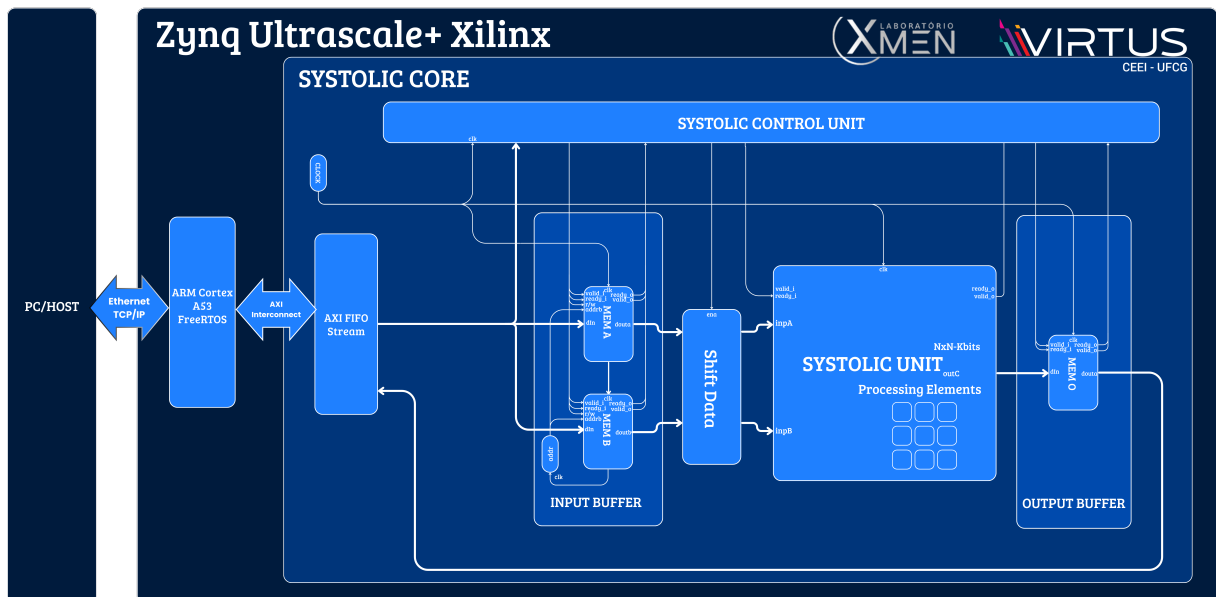
Versão	Plataforma	Interface de Comunicação	Principal Gargalo	Principal Melhoria Introduzida
0	Arty A7	UART entre host e acelerador	Baixa largura de banda da UART e limitação de recursos do FPGA, restringindo testes a matrizes pequenas	Validação funcional inicial do acelerador e identificação dos primeiros gargalos de comunicação e escalabilidade
1	Zynq ZCU106	Ethernet entre host e PS; AXI Stream FIFO entre PS e PL	Pré-processamento das matrizes no hospedeiro, elevado custo de comunicação e latência na transferência entre PS e PL	Migração para plataforma mais robusta, adoção de FreeRTOS, comunicação via Ethernet e integração com interfaces AXI de maior desempenho
2	Zynq ZCU106	Ethernet entre host e PS; AXI Stream FIFO entre PS e PL	Sobrecarga causada pelo envio de dados já deslocados para atender ao arranjo sistólico	Implementação em hardware do módulo de deslocamento das matrizes, eliminando o pré-processamento em software e reduzindo o custo de transferência de $N(2N - 1)$ para N^2
3	Zynq ZCU106	PS como host; DMA entre memória e PL	Sobrecarga da transferência mediada por FIFO e baixa eficiência no envio elemento a elemento	Integração de um controlador DMA e adoção da transferência em blocos de linhas das matrizes, reduzindo latência e aumentando a eficiência da comunicação

3.2 Diagrama em Blocos

O sistema é composto por cinco blocos principais: interface de entrada e saída, subsistema de memória, escalonador de matrizes, arranjo sistólico e unidade de controle. A

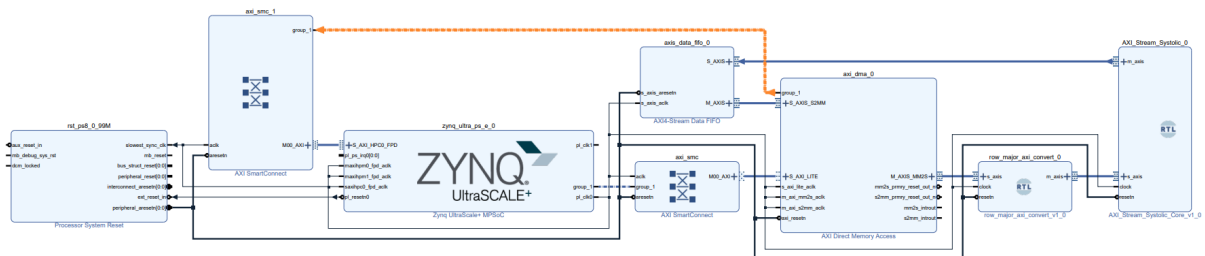
interface de entrada e saída realiza a comunicação com o sistema hospedeiro, transferindo as matrizes e resultados por meio de protocolos como DMA/AXI Stream. O subsistema de memória armazena temporariamente os dados, garantindo baixa latência e alimentação contínua do processamento. O escalonador organiza o fluxo de entrada, distribuindo os elementos das matrizes de forma adequada ao funcionamento do array. O arranjo sistólico constitui o núcleo computacional, formado por elementos de processamento que executam operações de multiplicação e acumulação de forma paralela. Por fim, a unidade de controle coordena o funcionamento do sistema, gerenciando sinais, sincronização, início e término das operações. A Figura 8 apresenta o diagrama em blocos geral da arquitetura proposta.

Figura 7 – Diagrama de Blocos do Sistema



Autoria: Própria.

Figura 8 – Diagrama de Blocos do Zynq UltraScale+ MPSoC



Autoria: Própria.

3.3 Fluxo de Dados do Acelerador

A estratégia de fluxo de dados adotada neste trabalho foi implementada em *streaming*, permitindo que os dados sejam transferidos progressivamente entre o Host, a parte programável por software (PS) e a parte programável por lógica (PL).

O processador ARM Cortex-A53 recebe os dados do Host e envia os dados de entrada ao acelerador por meio da interface **DMA/AXI Stream**, que desacopla o produtor (software) do consumidor (hardware). Inicialmente, os dados são armazenados em um *FIFO*, cuja função é absorver variações de latência e garantir alimentação contínua ao acelerador. Os dados são distribuídos em buffers dedicados às matrizes *A* e *B*, implementados como memórias internas do FPGA. A separação em dois bancos independentes permite a leitura simultânea dos operandos necessários em cada ciclo de computação, enquanto a escrita é realizada sequencialmente: primeiro a matriz *A* e depois a matriz *B*.

O bloco *Shift_Data* organiza os dados no arranjo sistólico seguindo um padrão temporal específico:

- Matrizes *A*: deslocamento horizontal ao longo das linhas;
- Matrizes *B*: deslocamento vertical ao longo das colunas.

Esse esquema assegura que, a cada ciclo de *clock*, os pares corretos de operandos se encontrem em cada unidade de processamento (*Processing Element*, PE) para execução de operações multiplicação-acumulação (*Multiply-Accumulate*, MAC). O arranjo permite reutilização de dados entre PEs vizinhos, reduzindo acessos repetidos à memória externa.

A unidade de controle sistólica gerencia sinais de habilitação, sincronização e validade (**valid/ready**), definindo as fases de operação do acelerador: carregamento dos dados, computação no array e drenagem dos resultados. Os resultados finais são armazenados em um buffer de saída e transferidos de volta ao Host via **AXI Stream/DMA**. Cabe ressaltar que, nessa etapa, há uma inversão no fluxo de dados produtor (hardware) e consumidor (software).

3.3.1 Parametrização do Arranjo Sistólico

O arranjo sistólico foi projetado com uma matriz bidimensional de $N \times N$, parametrizável em nível de descrição de hardware. Os dados são representados em inteiro de largura fixa, possibilitando adequar aos recursos disponíveis na FPGA e às exigências de precisão e da aplicação.

os principais parâmetros do projeto incluem:

- Tamanho do Arranjo Sistólico

- Tamanho do Símbolo

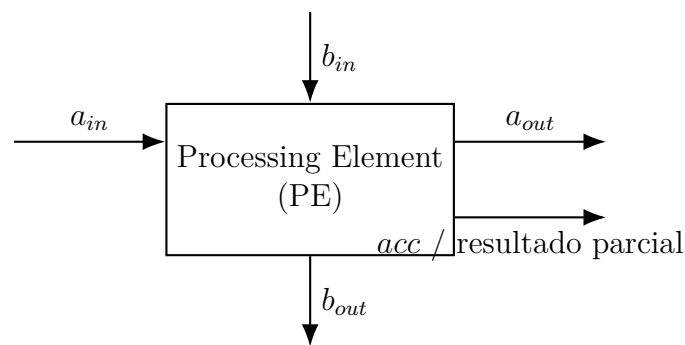
3.3.2 Elemento de Processamento (Processing Element – PE)

O *Processing Element* (PE) constitui a unidade básica de computação do arranjo sistólico. Cada PE é responsável por receber os dados de entrada provenientes dos elementos vizinhos ou dos registradores de borda, realizar a operação de multiplicação e acumulação correspondente e encaminhar os operandos para os próximos elementos da malha. Essa organização permite que o processamento ocorra de forma distribuída e sincronizada, explorando paralelismo espacial no cálculo da multiplicação de matrizes.

No contexto do acelerador proposto, cada PE recebe um elemento da matriz A e um elemento da matriz B , calcula o produto entre eles e acumula o resultado parcial internamente. Ao mesmo tempo, os operandos continuam sendo propagados para os PEs adjacentes, permitindo que diferentes produtos parciais sejam calculados em paralelo ao longo do arranjo. Dessa forma, o PE não atua de maneira isolada, mas como parte de uma estrutura regular e interconectada.

A Figura 9 apresenta uma visão funcional simplificada de um elemento de processamento, destacando os fluxos de entrada, processamento local e propagação de dados.

Figura 9 – Visão funcional de um elemento de processamento do arranjo sistólico.



Autoria: Própria.

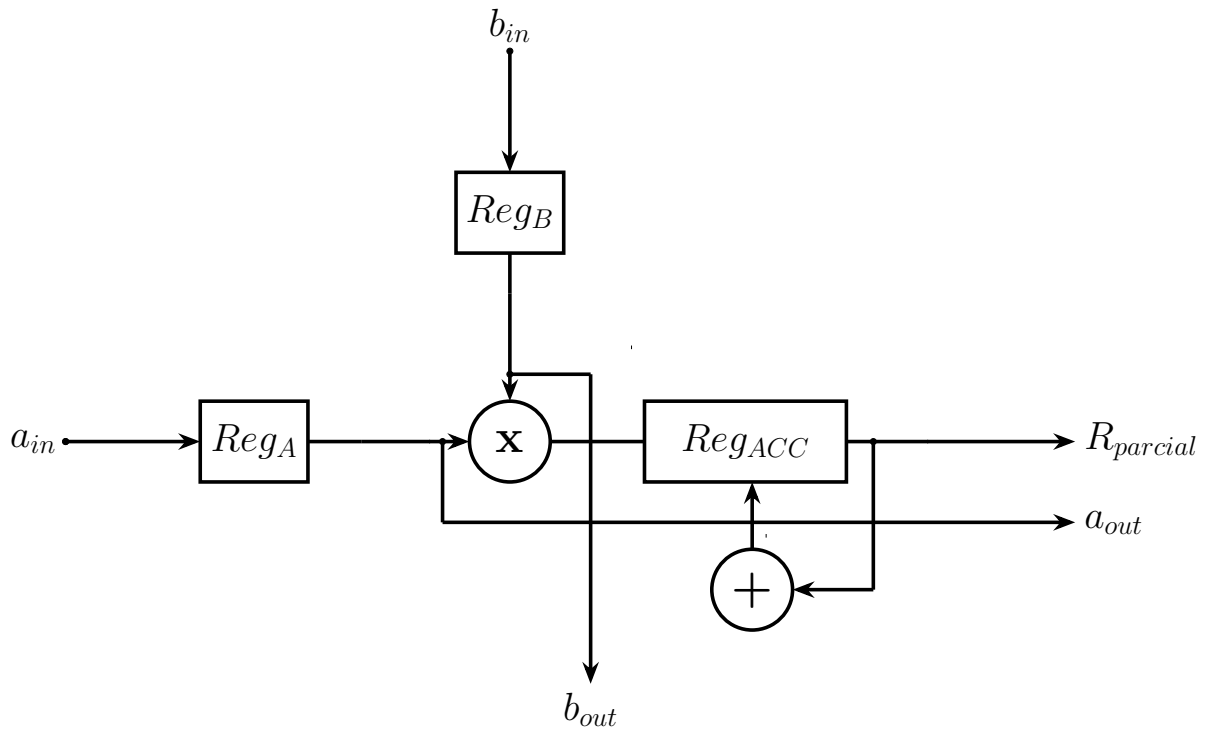
3.3.3 Estrutura Interna do PE

Internamente, o PE é composto por registradores de entrada, unidade de multiplicação, unidade de acumulação e lógica de repasse dos dados. Os operandos recebidos são sincronizados com o clock do sistema, de modo que a propagação pelo arranjo preserve o alinhamento temporal necessário ao funcionamento sistólico. O valor do acumulador interno armazena a soma parcial referente ao elemento de saída associado àquela posição do arranjo.

Essa estrutura permite que, a cada ciclo de clock, o PE execute simultaneamente três funções principais: receber novos dados, calcular um novo produto parcial e encaminhar os operandos para os PEs vizinhos. Como consequência, o arranjo mantém um fluxo contínuo de dados, reduzindo ociosidade e aumentando a eficiência do processamento matricial.

A Figura 10 ilustra a organização interna típica do PE utilizado na arquitetura proposta.

Figura 10 – Estrutura interna de um elemento de processamento.



Autoria: Própria.

3.3.4 Unidade Multiplicadora-Acumuladora (MAC)

A unidade *Multiply-Accumulate* (MAC) é o núcleo aritmético do PE. Sua função consiste em multiplicar os operandos de entrada e somar o produto obtido ao valor acumulado anteriormente. Em termos matemáticos, a operação realizada pode ser expressa por

$$acc_{k+1} = acc_k + (a \times b) \quad (3.1)$$

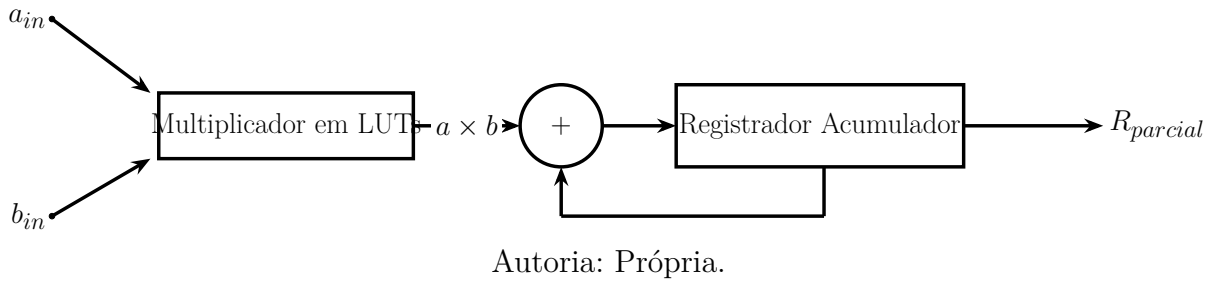
em que a e b representam os operandos correntes e acc_k corresponde ao valor acumulado até o ciclo anterior.

Na arquitetura desenvolvida, a operação MAC foi implementada sem o uso de blocos DSP dedicados do FPGA. Assim, tanto a multiplicação quanto a acumulação são

realizadas com recursos de lógica programável, principalmente LUTs e registradores. Essa escolha torna a arquitetura mais independente de recursos aritméticos especializados e permite avaliar com maior clareza o custo de escalabilidade do arranjo em termos de utilização de lógica.

A Figura 11 apresenta o diagrama conceitual da unidade MAC empregada no PE.

Figura 11 – Diagrama representativo do MAC.



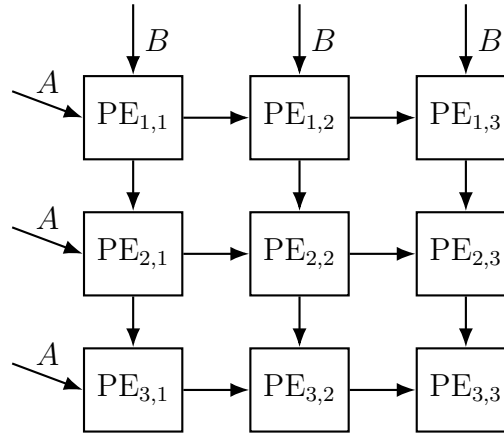
3.3.5 Comunicação e Propagação de Dados entre PEs

A comunicação entre os elementos de processamento segue o princípio clássico dos arranjos sistólicos: os dados fluem localmente entre vizinhos imediatos, enquanto os resultados parciais são acumulados ao longo do tempo no interior de cada PE. Em uma configuração típica para multiplicação de matrizes, os elementos da matriz A propagam-se horizontalmente ao longo das linhas do arranjo, enquanto os elementos da matriz B propagam-se verticalmente ao longo das colunas.

Esse esquema de comunicação local reduz a necessidade de interconexões globais complexas e favorece a modularidade da arquitetura. Além disso, como cada PE depende apenas dos dados recebidos dos vizinhos adjacentes, o arranjo torna-se naturalmente escalável. O escalonamento temporal das entradas garante que os pares corretos de operandos encontrem-se no instante adequado em cada posição da malha, produzindo os produtos parciais necessários ao cálculo do elemento correspondente da matriz resultante.

A Figura 12 ilustra a propagação dos operandos entre PEs em uma malha sistólica bidimensional.

Figura 12 – Comunicação local e propagação de dados entre elementos de processamento em um arranjo sistólico.



Autoria: Própria.

Observa-se que a movimentação dos dados ocorre de forma ritmada pelo clock, caracterizando o comportamento sistólico da arquitetura. A cada ciclo, novos dados entram nas bordas do arranjo, enquanto os dados previamente carregados avançam para os elementos vizinhos, permitindo a sobreposição entre comunicação e computação. Essa característica é fundamental para alcançar alto paralelismo com uma estrutura de controle relativamente simples.

3.3.6 Escalonamento das Entradas

As matrizes são injetadas no bloco responsável por escalonar os operandos, incorporando atrasos regulares e esparsos. Esse mecanismo garante que os dados cheguem sincronizados aos elementos de processamento, respeitando o padrão temporal exigido pela arquitetura.

O bloco possui dois parâmetros, $DATA_W$ e N_{LANES} , que representam, respectivamente, o tamanho do símbolo (quantidade de bits) a ser processado e a dimensão das matrizes a serem deslocadas. A interface do bloco contém os seguintes sinais: clock, nreset, habilitação, entrada e saída. O conjunto de entrada e saída é composto por um arranjo bidimensional formado por sinais de dados com largura de $DATA_W$ bits, replicados em múltiplas vias paralelas (N_{LANES}).

São instanciados registradores adicionais além do mínimo necessário, com o objetivo de preservar a regularidade estrutural. Essa abordagem evita lógica condicional assimétrica, melhora o roteamento na FPGA e simplifica a descrição parametrizável em RTL, contribuindo para maior escalabilidade e organização do projeto.

A quantidade total de registradores do deslocador cresce quadraticamente com a

dimensão do arranjo:

$$R(N) = N^2 \quad (3.2)$$

Já a latência estrutural do deslocador para se obter uma matriz totalmente deslocada é dada por:

$$Latência = N_{Preenchimento} + (N - 1)_{Diagonal} \quad (3.3)$$

De forma simplificada:

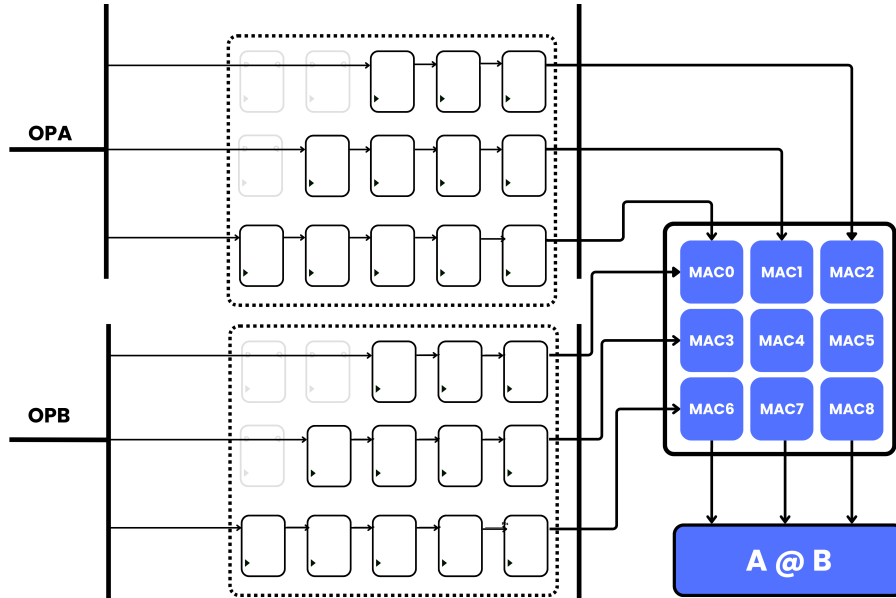
$$Latência(N) = 2N - 1 \quad (3.4)$$

Essa latência decorre do tempo necessário para preenchimento inicial do escalonador, propagação interna dos dados e escoamento final do pipeline, após o preenchimento inicial, o escalonador atinge o regime permanente, produzindo a aproximadamente N resultados por ciclo de clock, conforme expresso por:

$$Throughput_{esc} \approx N \text{ resultados por ciclo.} \quad (3.5)$$

A Figura 13 apresenta um diagrama em alto nível que ilustra o comportamento do escalonador de matrizes para um caso simplificado.

Figura 13 – Diagrama do Deslocador de Matrizes



Autoria: Própria.

Considere o exemplo de multiplicação de matrizes $A \in R^{3 \times 3}$ e $B \in R^{3 \times 3}$, as matrizes são injetadas no bloco escalonador de matrizes, onde as linhas de A e as colunas de B são deslocadas e alinhadas para que os elementos correspondentes sejam processados

simultaneamente pelos PEs do arranjo sistólico. O processo de alinhamento é ilustrado a seguir, onde os zeros representam os atrasos incorporados para garantir o fluxo regular dos dados.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \quad \text{e} \quad B = \begin{bmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix} \quad (3.6)$$

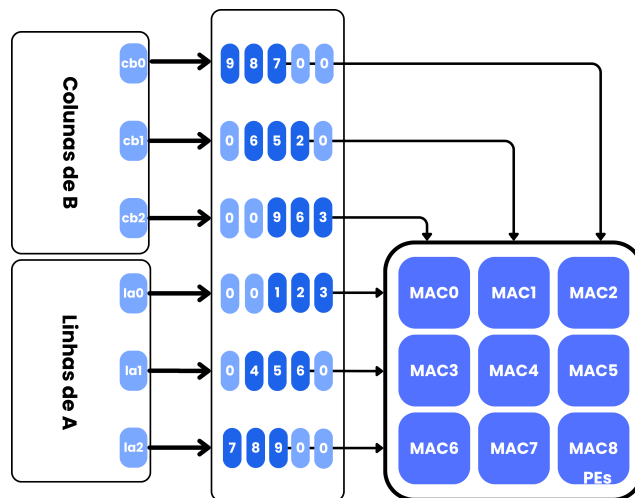
$$B^T = \begin{bmatrix} 9 & 6 & 3 \\ 8 & 5 & 2 \\ 7 & 4 & 1 \end{bmatrix} \quad (3.7)$$

Tabela 2 – Evolução temporal do escalonador de matrizes para A e B

A						B					
Pipeline	C4	C3	C2	C1	C0	Pipeline	C4	C3	C2	C1	C0
A0	0	0	1	2	3	B0	0	0	9	6	3
A1	0	4	5	6	0	B1	0	8	5	2	0
A2	7	8	9	0	0	B2	7	4	1	0	0

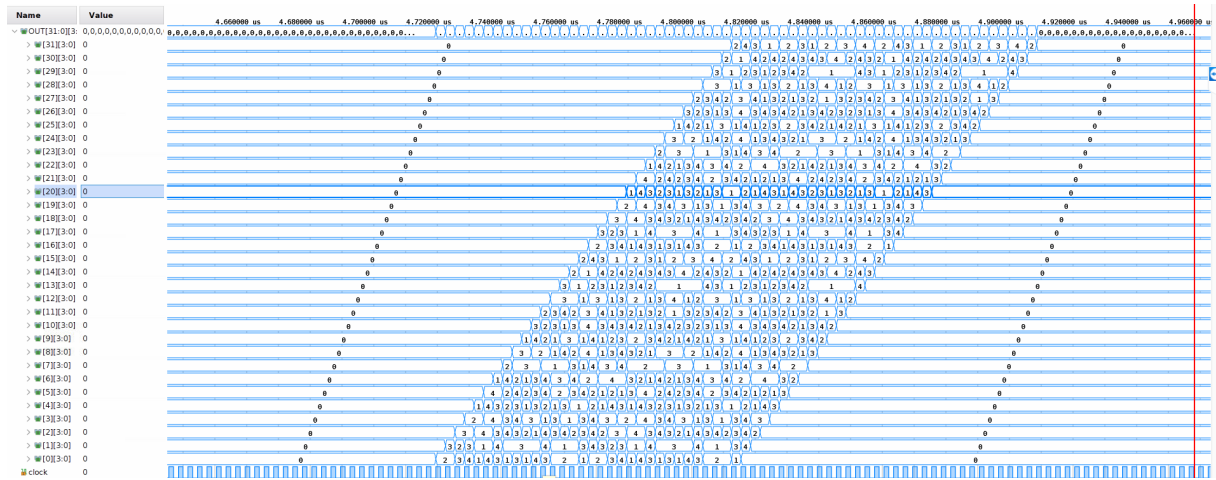
Observa-se, na Tabela 2, que o primeiro dado válido atravessa o deslocador em cinco ciclos, corroborando a expressão 3.4. A Figura 14 apresenta a integração do deslocador ao subsistema de memória e à unidade de arranjo sistólico, com o pipeline completamente preenchido considerando as matrizes 3.6 e 3.7. A Figura 15 mostra a simulação do escalonador para uma matriz 32x32 com 4 bits por símbolo.

Figura 14 – Exemplo do Escalonador de Matrizes



Autoria: Própria.

Figura 15 – Simulação do Escalonador de Matrizes 32x32-4 bits por símbolo



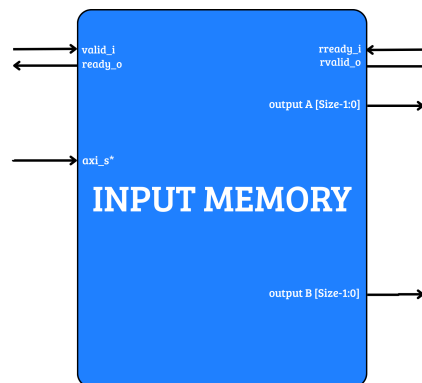
Autoria: Própria.

3.4 Subsistemas e Controle Geral

3.4.1 Subsystema de Memória de Entrada

O subsistema de memória de entrada é responsável por receber, armazenar e disponibilizar os operandos utilizados pelo acelerador. Conforme ilustrado na Figura 8, o módulo recebe os dados provenientes do processador embarcado por meio de uma interface DMA/AXI Stream e os armazena em memória interna, organizando-os de forma adequada para o consumo pelo núcleo de processamento. A comunicação com os demais módulos é realizada por meio de sinais de controle baseados no protocolo *valid/ready*, garantindo a sincronização entre produtor e consumidor de dados.

Figura 16 – Bloco do Subsystema de Entrada



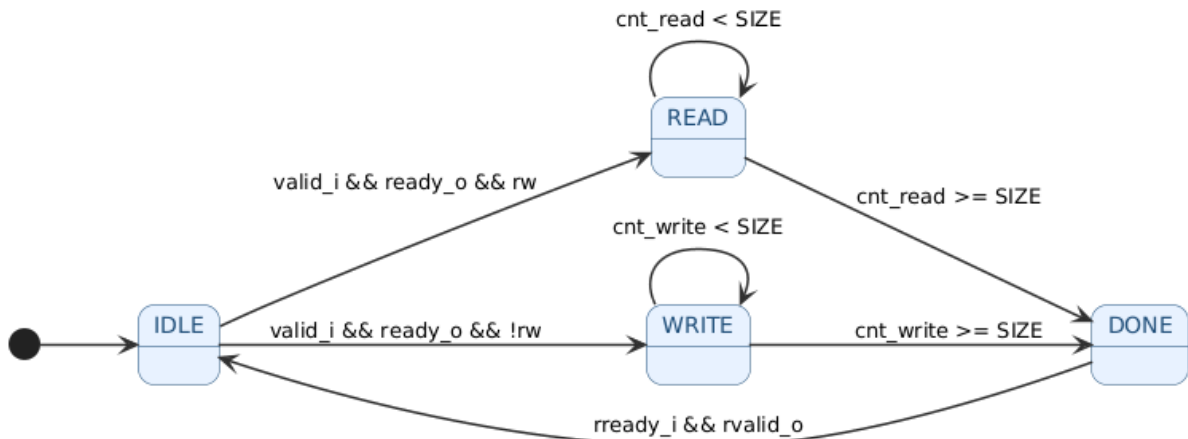
Autoria: Própria.

Os operandos correspondem às matrizes A e B , que são recebidas sequencialmente da memória principal. Inicialmente, os dados são transferidos linha a linha e armazenados no subsistema de memória de entrada. Durante essa fase, o sistema permanece no estado de **escrita**, no qual os dados recebidos pela interface DMA/AXI Stream são armazenados sequencialmente na memória interna. Nesse processo, primeiro são armazenadas todas as linhas da matriz A e, em seguida, todas as linhas da matriz B . Essa estratégia simplifica o controle de acesso à memória e permite aproveitar transferências em rajada (*burst*) da interface DMA/AXI Stream, aumentando a eficiência da transferência de dados.

Após o armazenamento completo dos operandos, inicia-se a fase de **leitura**, na qual os dados são disponibilizados simultaneamente para o acelerador. Nesse estágio, as matrizes A e B são fornecidas por meio das saídas **output A** e **output B**, permitindo que ambos os operandos sejam acessados em paralelo pelo núcleo de processamento. Essa leitura simultânea é fundamental para garantir que o acelerador receba continuamente os dados necessários para o processamento, evitando interrupções no fluxo de dados e garantindo o aproveitamento do paralelismo da arquitetura.

A Figura 17 apresenta a máquina de estados responsável pelo controle do subsistema. O sistema inicia no estado **IDLE**, aguardando a disponibilidade de dados na interface de entrada. Quando os sinais **valid_i**, **ready_o** e **!rw** são ativados, o sistema transita para o estado **WRITE**, no qual os dados recebidos são armazenados sequencialmente na memória interna. Após o preenchimento completo das posições de memória necessárias, o sistema transita para o estado **IDLE**, e espera até que os sinais **valid_i**, **ready_o** e **rw** sejam ativos para então transicionar para o estado **READ**, responsável por fornecer simultaneamente os operandos A e B ao acelerador. Quando a leitura dos dados é concluída, o sistema retorna ao estado **IDLE**, ficando pronto para uma nova operação.

Figura 17 – Máquina de Estados do Subsistema de Memória de Entrada



Autoria: Própria.

Outro aspecto importante desse subsistema é a largura de banda necessária para alimentar o acelerador. Considerando que os elementos da matriz são transmitidos em formato vetorial, a largura de banda necessária para cada linha é dada pela Equação 3.8, onde $SIZE$ corresponde ao número de elementos da linha e $WIDTH$ representa a largura em bits de cada operando.

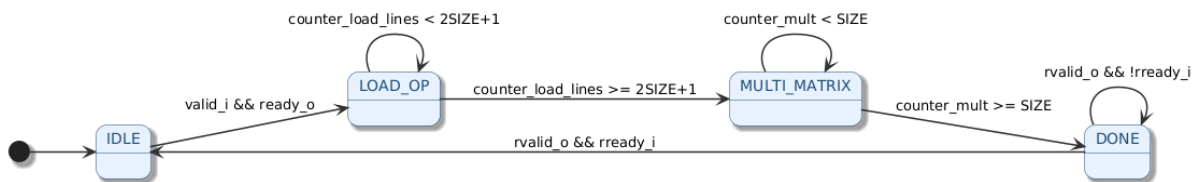
$$BW_{input} = SIZE \times WIDTH \quad (3.8)$$

Como os dados são recebidos da memória externa linha a linha, a interface de comunicação deve ser capaz de sustentar essa taxa de transferência para evitar gargalos no sistema. Dessa forma, a largura de banda necessária para transferir uma linha completa da matriz é proporcional ao produto entre o tamanho do elemento e o número de elementos por linha. Essa abordagem foi adotada em virtude dos problemas e gargalos observados anteriormente entre a transmissão PS-PL, que provocavam uma grande sobrecarga de comunicação e aumento da latência na alimentação do acelerador.

3.4.2 Unidade de Arranjo Sistólico

A unidade de arranjo sistólico é responsável pela execução do cálculo da multiplicação de matrizes utilizando um conjunto organizado de elementos de processamento (*Processing Elements* – PEs) conectados de forma regular. Essa organização explora paralelismo espacial e temporal, permitindo que múltiplas operações de multiplicação e acumulação sejam realizadas simultaneamente ao longo da matriz de processamento.

Figura 18 – Máquina de Estados do Subsistema de Cálculo Matricial



Autoria: Própria.

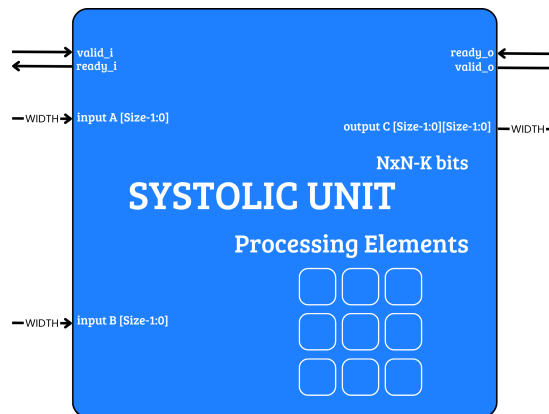
A Figura 18 apresenta a máquina de estados responsável pelo controle do subsistema de cálculo matricial. O funcionamento da unidade inicia-se no estado *IDLE*, no qual o sistema permanece aguardando a chegada de dados válidos provenientes do subsistema de memória de entrada. Quando os sinais de controle indicam que os operandos estão disponíveis e que o módulo está pronto para receber dados, ocorre a transição para o estado *LOAD_OP*.

No estado *LOAD_OP*, os operandos são lidos dos registradores internos e alinhados por meio do bloco responsável por deslocar as matrizes, que garante o sincronismo necessário para o correto fluxo de dados no arranjo sistólico. Durante essa etapa, os primeiros dados começam a se propagar pelo arranjo de elementos de processamento, permitindo que as operações de multiplicação e acumulação sejam iniciadas de forma sobreposta ao processo de carregamento.

Após o carregamento inicial dos operandos, o sistema transiciona para o estado *MULT_MATRIX*, no qual aguarda a propagação completa dos dados na malha de arranjo sistólico. O controle da duração dessas etapas é realizado por meio de contadores interno, responsável por acompanhar o progresso do fluxo de dados no arranjo sistólico. Esses contadores são utilizados durante o estado de carregamento e durante o estado de processamento, permitindo determinar quando os dados foram completamente inseridos e propagados pelo arranjo de elementos de processamento.

Quando o contador atinge o valor correspondente ao número total de ciclos necessários para a propagação completa dos dados, o sistema transita para o estado *DONE*, indicando que o cálculo foi finalizado e que os resultados podem ser disponibilizados ao subsistema de saída. Após a leitura dos resultados, o sistema retorna ao estado *IDLE*, ficando pronto para iniciar um novo ciclo de processamento.

Figura 19 – Bloco do Subsistema da Unidade de Arranjo Sistólico



Autoria: Própria.

A Figura 19 apresenta o diagrama estrutural da unidade do arranjo sistólico. O bloco principal, denominado *Systolic Unit*, contém uma matriz de elementos de processamento organizada em uma estrutura bidimensional $N \times N$. Cada elemento de processamento realiza operações de multiplicação e acumulação e repassa os dados para os elementos vizinhos, permitindo a propagação dos operandos ao longo da matriz.

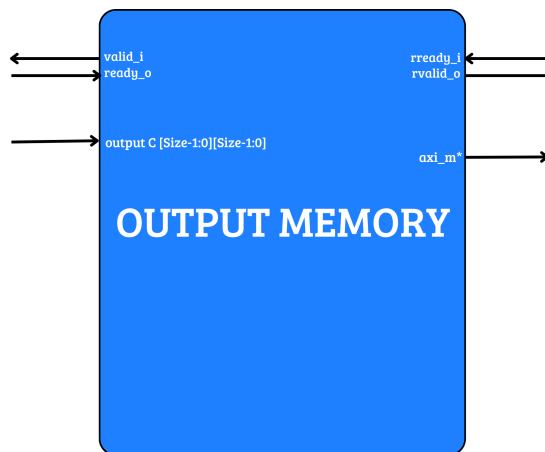
Os dados de entrada são fornecidos por meio de barramentos dedicados para os

operandos das matrizes, enquanto sinais de controle como *valid* e *ready* são utilizados para implementar o protocolo de comunicação entre os módulos. Esse mecanismo garante que o fluxo de dados ocorra de forma sincronizada, evitando perdas de dados.

3.4.3 Subsistema de Memória de Saída

O subsistema de memória de saída é responsável por receber os resultados produzidos pela unidade de arranjo sistólico, armazená-los temporariamente e organizá-los de forma compatível com a interface de comunicação utilizada para a transferência ao sistema hospedeiro. Conforme ilustrado na Figura 20, esse bloco atua como a interface entre o núcleo de processamento matricial e o barramento DMA/AXI Stream de saída, realizando a adaptação entre o formato interno dos dados calculados e o formato de transmissão exigido pelo sistema.

Figura 20 – Bloco do Subsistema de Saída



Autoria: Própria.

Diferentemente de uma estratégia em que cada elemento da matriz resultante seria transmitido individualmente, nesta arquitetura os dados são agrupados em *linhas completas* antes de serem enviados. Assim, a cada operação de leitura da memória de saída, uma linha inteira da matriz resultado C é empacotada em uma única palavra de dados e disponibilizada na interface DMA/AXI Stream.

Considerando uma matriz quadrada de dimensão $SIZE \times SIZE$, em que cada elemento possui largura de $WIDTH$ bits, cada linha contém $SIZE$ elementos. Dessa forma, a largura total do barramento necessária para transmitir uma linha completa em uma única transferência é dada pela expressão 3.9 em que BW_{output} representa a largura, em bits, da palavra transmitida a cada batimento da interface DMA/AXI Stream. Essa

organização permite que cada linha da matriz resultado seja enviada em um único ciclo de transferência válido, desde que o canal de comunicação esteja apto a aceitar os dados.

$$BW_{output} = SIZE \times WIDTH \quad (3.9)$$

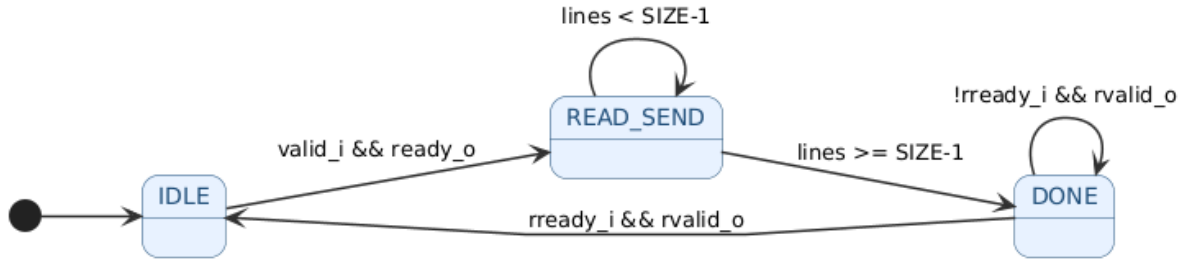
Essa escolha arquitetural é importante para manter compatibilidade com a taxa de produção do arranjo sistólico e evitar que a etapa de escoamento dos resultados se torne um gargalo no sistema. Como o núcleo computacional gera os elementos da matriz de saída de forma estruturada, o agrupamento por linhas reduz a complexidade de empacotamento no estágio final e simplifica a reconstrução da matriz no lado receptor. Além disso, essa estratégia melhora a eficiência de utilização do barramento, reduzindo a quantidade de transações necessárias quando comparada à transmissão elemento a elemento.

Internamente, o subsistema armazena os valores produzidos pelo arranjo sistólico em uma memória ou banco de registradores organizado de modo que os elementos pertencentes à mesma linha possam ser lidos de forma conjunta. Após o cálculo de uma linha, ou quando os dados correspondentes estão disponíveis, o bloco concatena os $SIZE$ elementos daquela linha em uma única palavra de $SIZE \times WIDTH$ bits, compatível com a interface DMA/AXI Stream mestre.

O controle da transmissão é realizado por meio dos sinais do protocolo *valid/ready*, garantindo a sincronização entre produtor e consumidor de dados. Quando o subsistema possui uma linha pronta para envio, o sinal `rvalid_o` é ativado, indicando que os dados presentes no barramento são válidos. A transferência ocorre apenas quando o módulo receptor sinaliza disponibilidade por meio de `rready_i`. Dessa forma, a movimentação dos dados segue o mecanismo de *handshake* do protocolo, assegurando robustez mesmo em situações de *backpressure*. Enquanto `rready_i` permanecer desativado, o subsistema mantém os dados estáveis na saída e preserva `rvalid_o` ativo até que a transferência seja concluída.

A máquina de estados apresentada na Figura 21 descreve o funcionamento do subsistema. No estado *IDLE*, o módulo permanece em espera até que haja dados válidos provenientes da unidade de processamento. Quando uma matriz resultado está pronta e as condições de transmissão são satisfeitas, ocorre a transição para o estado *READ_SEND*. Nesse estado, as linhas lidas da memória interna, serão empacotadas em uma palavra de $SIZE \times WIDTH$ bits e apresentada na interface DMA/AXI Stream de saída.

Figura 21 – Máquina de Estados do Subsistema de Memória de Saída



Autoria: Própria.

Enquanto ainda existirem linhas a serem transmitidas, o subsistema permanece no estado *READ_SEND*, incrementando o índice de linha a cada transferência concluída. Para uma matriz de dimensão $SIZE \times SIZE$, são necessárias $SIZE$ transferências para o envio completo da matriz resultado, sendo cada transferência correspondente a uma linha inteira.

Quando o contador de linhas atinge o valor limite, indicando que a última linha foi transmitida, a máquina de estados transita para o estado *DONE*. Nesse estado, o subsistema sinaliza a conclusão da operação e aguarda o encerramento da comunicação antes de retornar ao estado *IDLE*, tornando-se apto para iniciar uma nova operação de processamento.

Do ponto de vista de desempenho, essa organização impõe um requisito importante ao canal de saída: a interface deve ser capaz de sustentar uma palavra de $SIZE \times WIDTH$ bits por transferência para que cada linha seja enviada sem fragmentação. Caso a largura física do barramento seja inferior a esse valor, torna-se necessário fragmentar cada linha em múltiplos batimentos DMA/AXI Stream, aumentando o número de ciclos de transmissão e potencialmente reduzindo o *throughput* do acelerador.

Portanto, o agrupamento dos dados por linhas permite alinhar a organização interna da matriz com o mecanismo de transmissão, reduzindo a sobrecarga de comunicação e favorecendo uma utilização mais eficiente do barramento DMA/AXI Stream.

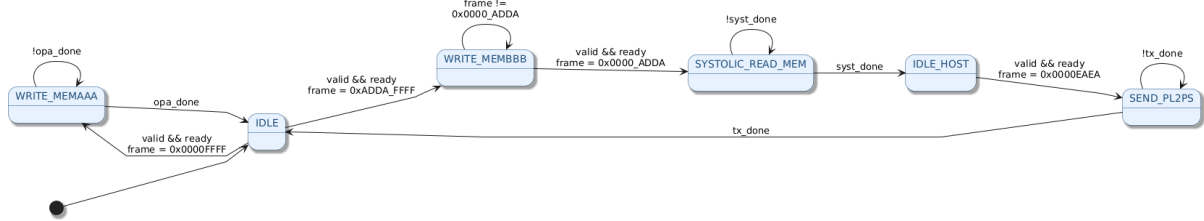
3.4.4 Unidade de Controle e Sequenciamento Operacional

A unidade de controle é responsável por coordenar o funcionamento global do acelerador, gerenciando o fluxo de dados entre os módulos de comunicação, memória e o arranjo sistólico responsável pelo processamento, interpretar comandos recebidos pela interface de comunicação e acionar os diferentes subsistemas do acelerador.

A Figura 22 apresenta o diagrama de estados da unidade de controle. O sistema inicia sua operação no estado *IDLE*, no qual permanece aguardando a recepção de um

quadro de comando válido proveniente da interface de comunicação DMA/AXI Stream. Durante esse estado, os bytes recebidos são acumulados em um registrador denominado *frame_start*, permitindo a identificação de padrões específicos de comando.

Figura 22 – Máquina de Estados da Unidade de Controle Geral



Autoria: Própria.

Quando a sequência de dados recebida corresponde ao padrão de inicialização de transferência do operando A (0x0000_FFFF), a máquina de estados realiza a transição para o estado *WRITE_MEMAAA*. Nesse estado, o subsistema de memória recebe os dados do operando A, que são armazenados sequencialmente na memória interna associada ao acelerador.

Após a conclusão da escrita do operando A, indicada pelo sinal *opa_done*, a FSM retorna temporariamente ao estado *IDLE*. Em seguida, ao identificar o comando correspondente à transmissão do operando B (0x0000_FFFF), a unidade de controle realiza a transição para o estado *WRITE_MEMBBB*, no qual os elementos da segunda matriz são armazenados na memória interna do sistema.

Uma vez que ambos os operandos tenham sido carregados, o sistema aguarda o comando que indica o início do processamento. Ao identificar o quadro de comando correspondente (0x0000_ADDA), a máquina de estados transita para o estado *SYSTOLIC_READ_MEM*. Nesse estágio, os dados armazenados nas memórias internas passam a ser fornecidos ao arranjo sistólico, iniciando a execução da operação de multiplicação de matrizes.

Após a conclusão do processamento no arranjo sistólico, indicada pelo sinal *syst_done*, o sistema entra no estado *IDLE_HOST*, no qual aguarda um comando externo para iniciar a transmissão dos resultados.

Finalmente, quando o comando de transmissão é identificado (0x0000_EAEA), a FSM transita para o estado *SEND_PL2PS*, responsável por transferir os resultados do acelerador para o sistema hospedeiro. Ao término dessa transmissão, indicado pelo sinal *tx_done*, o sistema retorna ao estado *IDLE*, concluindo o ciclo completo de operação do acelerador.

Essa organização baseada em FSM permite o controle determinístico do fluxo de dados e a sincronização adequada entre os diferentes módulos do sistema, garantindo que as etapas de carregamento, processamento e transmissão ocorram de forma ordenada e sem conflitos de acesso aos recursos internos.

4 Simulação e Análises

4.1 Verificação Funcional

A verificação funcional do acelerador foi realizada por meio de simulações em nível de RTL (*Register Transfer Level*). O objetivo desta etapa foi garantir que o comportamento do hardware implementado corresponde ao modelo funcional esperado.

Durante o processo de verificação, foram utilizados testbenches desenvolvidos em SystemVerilog para aplicar estímulos ao circuito e observar as respostas produzidas pela arquitetura. Os resultados gerados pelo acelerador foram comparados com valores de referência obtidos por meio de uma implementação equivalente em software.

4.1.1 Metodologia de Teste

A metodologia de verificação funcional adotada baseou-se na aplicação de vetores de teste representativos, com o objetivo de validar o correto funcionamento dos principais módulos da arquitetura. Inicialmente, foram definidos casos de teste contendo diferentes matrizes de entrada, incluindo valores aleatórios e padrões específicos projetados para exercitar diferentes caminhos de execução do hardware. Esses vetores foram inseridos no sistema por meio do testbench, que também foi responsável por capturar as saídas produzidas pelo acelerador.

Os resultados obtidos foram comparados com aqueles gerados por uma implementação de referência em software, garantindo a equivalência funcional entre ambas as abordagens. Adicionalmente, foram monitorados sinais internos relevantes durante a simulação, permitindo verificar o comportamento temporal dos módulos e confirmar a correta propagação dos dados através da arquitetura.

4.1.2 Resultados de Simulação

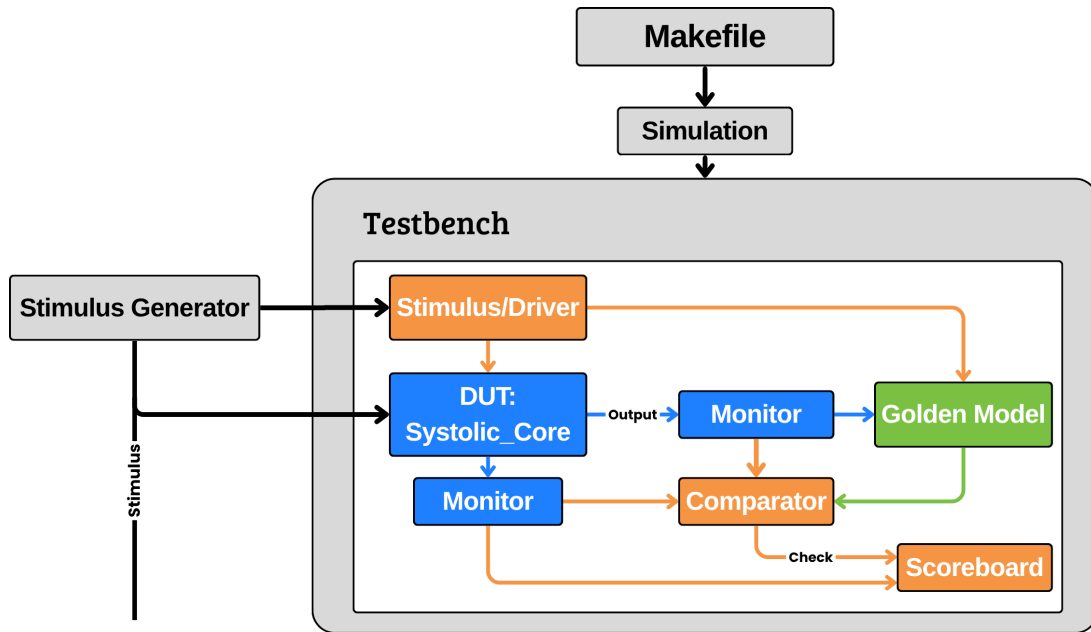
Nas simulações realizadas, consideraram-se os seguintes tamanhos de arranjos sistólicos: 2×2 , 4×4 , 16×16 , 32×32 , 64×64 e, por fim, 128×128 , utilizando 4 bits por símbolo de entrada e 8 bits por símbolo de saída em todos os casos avaliados. A partir de 256×256 , o tempo necessário para simulação e síntese tornou-se significativamente elevado.

Os resultados gerados pelo hardware apresentaram correspondência exata com os valores obtidos pela implementação em software, confirmando a correção funcional do sistema. Além disso, as formas de onda obtidas durante a simulação permitiram verificar a

sincronização adequada entre os diferentes módulos da arquitetura. A Figura 23 ilustra o ambiente de verificação do DUT.

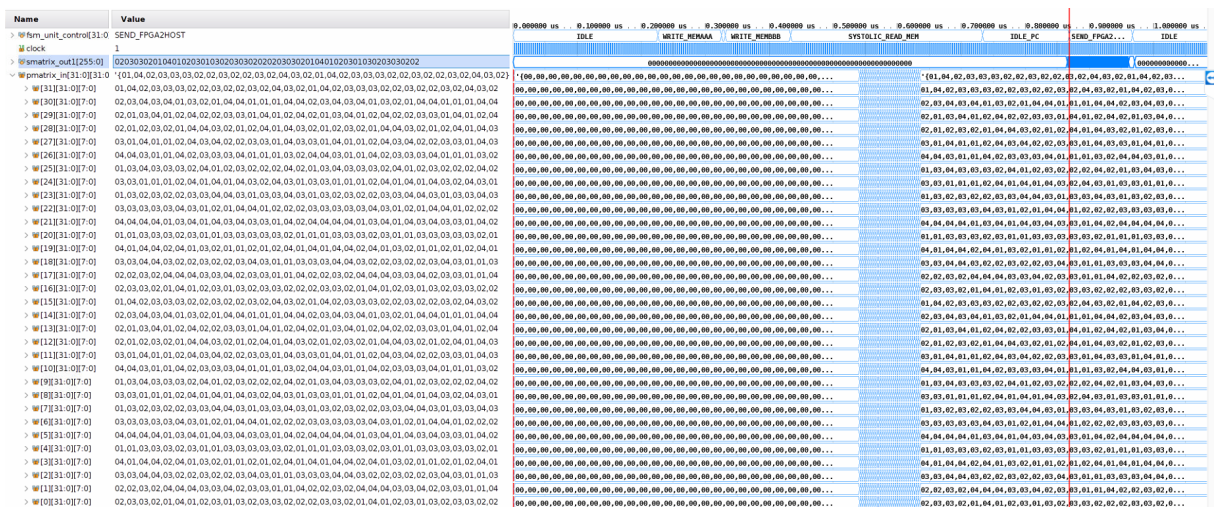
Esses resultados indicam que o acelerador foi corretamente implementado em nível de RTL, possibilitando avançar para as etapas de síntese e avaliação de desempenho.

Figura 23 – Diagrama de Blocos do Ambiente de Verificação do Acelerador Sistólico



Autoria: Própria.

Figura 24 – Simulação de uma Multiplicação $I \times B = B$, 32×32 - 4 Bits por Símbolo.



Autoria: Própria.

4.1.3 Latência

A latência corresponde ao número de ciclos de clock necessários para que uma operação completa seja realizada desde a entrada dos dados até a geração do resultado final. A Tabela 3 apresenta a latência em ciclos de clock das principais etapas do acelerador para diferentes versões da arquitetura. Observa-se que a versão 1.3 reduz significativamente a latência das etapas de transferência de dados, passando de uma complexidade quadrática $O(N^2)$ para uma complexidade linear $O(N)$ tabela 4.

Tabela 3 – Latência observada nas simulações em ciclos de clock para cada etapa do acelerador nas diferentes versões da arquitetura

Versão	Escrita na Memória (A e B)	Computação do Arranjo Sistólico	Transferência para o Host	Latência Fixa
Versão inicial 1.0	$2N(2N - 1)$	$3N$	N^2	15
Otimização Parcial 2.0	$2N^2$	$3N$	N^2	15
Arquitetura Otimizada 3.0	$2N$	$3N$	N	15

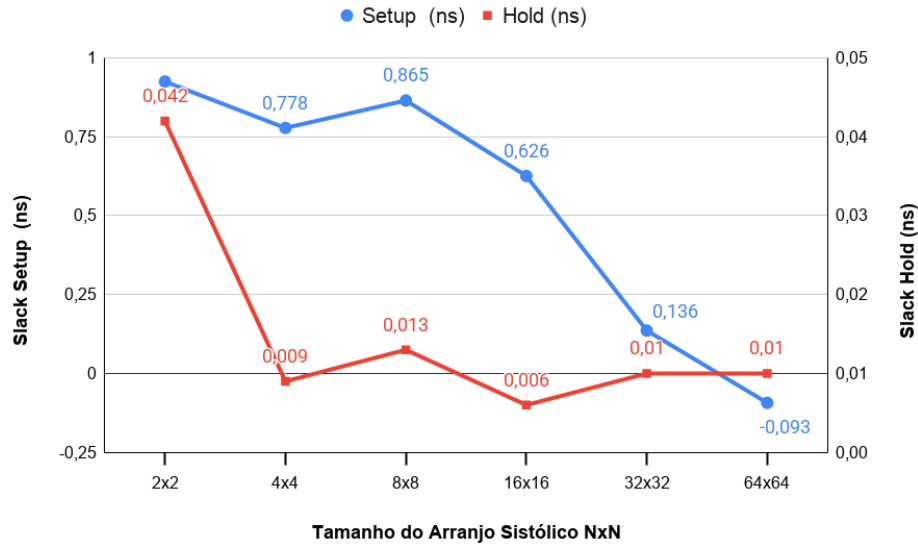
Tabela 4 – Evolução da arquitetura do acelerador

Versão	Latência Total (ciclos)	Complexidade	Observação
1.0	$5N^2 + N + 15$	$O(N^2)$	Arquitetura inicial com matrizes previamente organizadas em memória.
2.0	$3N^2 + 3N + 15$	$O(N^2)$	Introdução de um escalonador para organizar o fluxo de dados, reduzindo dependências no envio das matrizes ao arranjo.
3.0	$6N + 15$	$O(N)$	Uso de escalonador combinado com envio estruturado de linhas, permitindo a redução significativa da latência.

4.1.4 Análise de Temporização (Timing)

A análise de temporização foi realizada a partir dos relatórios de *Static Timing Analysis* (STA), considerando uma frequência de operação de 333 MHz, correspondente a um período de clock de aproximadamente 3 ns. Foram avaliados os valores de *setup slack* e *hold slack* para diferentes tamanhos do arranjo sistólico.

Figura 25 – Slack de setup e hold em função do tamanho do arranjo sistólico a @333 MHz.



Autoria: Própria.

Como mostrado na Figura 25, os arranjos menores (2×2 até 16×16) apresentam *setup slack* positivo, indicando que o circuito atende às restrições temporais para a frequência analisada. O maior valor ocorre no arranjo 8×8 , com aproximadamente 0,865 ns de margem temporal.

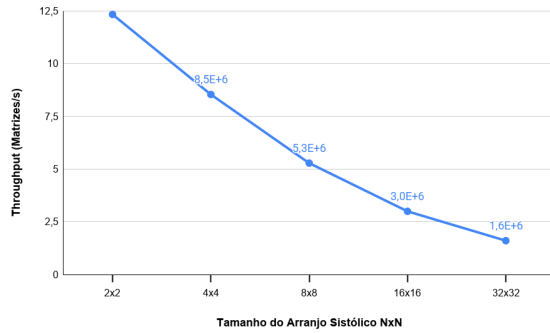
À medida que o tamanho do arranjo aumenta, observa-se redução do *setup slack*, resultado do aumento do caminho crítico causado pelo crescimento do número de elementos de processamento e das interconexões. No arranjo 64×64 ocorre uma violação de temporização, com *setup slack* de -0,093 ns, indicando que o circuito não atende às restrições de temporização para 333 MHz nessa configuração.

Por outro lado, os valores de *hold slack* permanecem positivos em todas as configurações avaliadas, indicando ausência de violações de *hold*. Esses resultados evidenciam que o aumento do tamanho do arranjo sistólico impacta diretamente o caminho crítico do circuito e a margem temporal disponível.

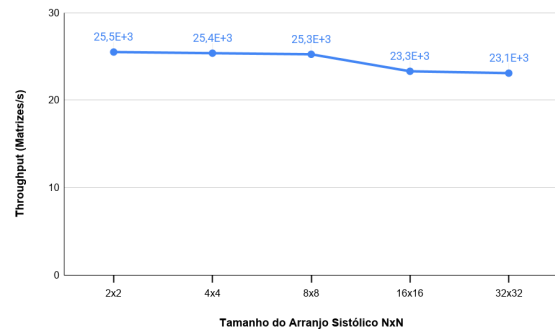
4.1.5 Throughput Teórico x Mensurado

A Figura 26(a) apresenta o *throughput* teórico do acelerador para diferentes tamanhos de arranjos sistólicos $N \times N$, enquanto a Figura 26(b) apresenta o *throughput* efetivamente mensurado durante os experimentos utilizando a interface de comunicação baseada em AXI DMA.

Figura 26 – Comparação entre throughput teórico (matrizes/s) e throughput medido (matrizes/s) para diferentes tamanhos de arranjos sistólicos.



((a)) Throughput teórico do acelerador.



((b)) Throughput medido considerando comunicação via AXI DMA.

Autoria: Própria.

No caso do *throughput* teórico, a métrica representa a capacidade máxima de processamento do arranjo sistólico considerando apenas a frequência de operação do hardware e o paralelismo intrínseco da arquitetura. Os valores estão expressos em **milhões de matrizes por segundo (Mmatrizes/s)** e indicam que arranjos menores conseguem processar um maior número de matrizes completas por segundo, uma vez que o tempo necessário para preencher e esvaziar o *pipeline* cresce com o tamanho do arranjo.

Por outro lado, os resultados apresentados na Figura 26(b), expressos em **milhares de matrizes por segundo (kmatrizes/s)**, mostram uma degradação significativa quando comparados aos valores teóricos. Enquanto o modelo ideal indica valores na ordem de milhões de matrizes por segundo, o desempenho experimental obtido encontra-se na ordem de dezenas de milhares de matrizes por segundo.

Essa diferença ocorre principalmente devido ao **overhead de comunicação introduzido pelo mecanismo de transferência de dados via DMA**. No cenário experimental, cada multiplicação de matriz requer a transferência das matrizes de entrada da memória do sistema (PS) para a lógica programável (PL) e posteriormente a transferência do resultado de volta para a memória. Esse processo envolve operações de configuração do DMA, latência de inicialização da transação e sincronização entre processador e acelerador.

Como consequência, o tempo total de execução passa a ser dominado não apenas pelo tempo de computação do arranjo sistólico, mas também pelo custo de movimentação de dados. Esse efeito torna-se particularmente evidente para matrizes pequenas, nas quais o tempo de cálculo é muito reduzido quando comparado ao tempo de comunicação.

Uma estratégia para reduzir esse impacto consiste em **aumentar o número de matrizes processadas por transação de DMA**. Em vez de transferir e processar uma única multiplicação por vez, múltiplas matrizes podem ser previamente armazenadas em

blocos maiores de memória e enviadas ao acelerador em fluxo contínuo. Essa abordagem permite amortizar o custo de inicialização do DMA ao longo de diversas operações, aproximando o desempenho observado do limite teórico do hardware.

Outra possibilidade consiste na utilização de técnicas de *buffering* e *tiling* de dados, nas quais conjuntos de matrizes são mantidos em memórias locais (como BRAM ou URAM) próximas ao acelerador. Dessa forma, o arranjo sistólico pode operar continuamente sobre grandes volumes de dados sem a necessidade de realizar transferências frequentes entre PS e PL.

Portanto, embora o desempenho mensurado esteja significativamente abaixo do limite teórico apresentado na Figura 26(a), os resultados indicam que o gargalo principal do sistema está relacionado à largura de banda e à latência de comunicação, e não à capacidade computacional do arranjo sistólico. A adoção de estratégias de processamento em lote e melhor exploração da hierarquia de memória tende a reduzir esse efeito e aumentar o *throughput* efetivo do acelerador.

4.1.6 Utilização do FPGA

A Tabela 5 apresenta a utilização dos principais recursos lógicos do FPGA para diferentes tamanhos do arranjo sistólico implementado. Foram analisados três tipos principais de recursos da arquitetura da FPGA: *Look-Up Tables* (LUTs), registradores (*Registers*) e *Configurable Logic Blocks* (CLBs).

Tabela 5 – Utilização de recursos do FPGA para diferentes tamanhos do arranjo sistólico.

Arranjo	Recursos Utilizados			Utilização (%)		
	LUTs	Registers	CLB	LUTs	Registers	CLB
2x2	326	289	66	0,1%	0,1%	0,1%
4x4	898	782	175	0,1%	0,2%	1,1%
8x8	3148	2635	579	1,1%	0,6%	2,1%
16x16	11459	9835	2151	5,1%	2,1%	7,1%
32x32	44095	38036	7996	19,1%	8,3%	28,1%
64x64	174060	149844	27521	76,1%	32,5%	96,1%

A Figura 27(b) mostra a utilização percentual desses recursos em função do tamanho do arranjo sistólico. Observa-se que, para arranjos pequenos, como 2×2 e 4×4 , a ocupação de recursos é praticamente desprezível, permanecendo abaixo de 0,2% para LUTs e registradores. Mesmo para um arranjo 8×8 , a utilização ainda permanece relativamente baixa, atingindo apenas 1,1% das LUTs e 2,1% dos CLBs.

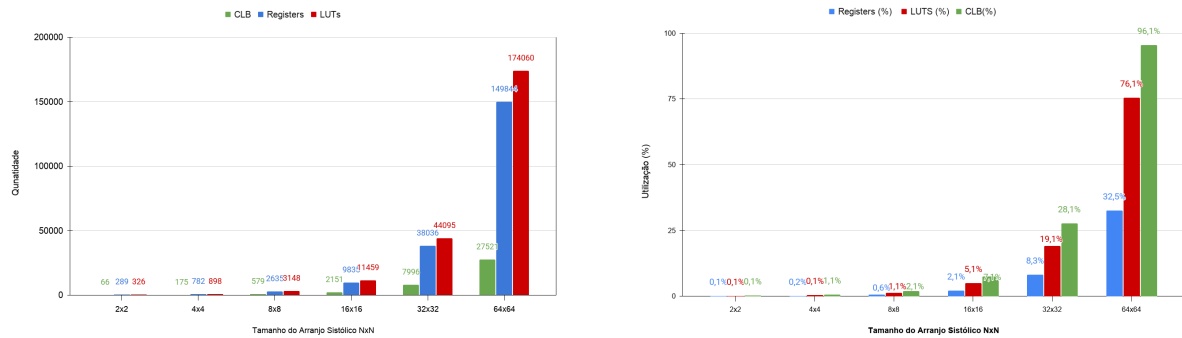
À medida que o tamanho do arranjo aumenta, o consumo de recursos cresce de forma significativa. Para o arranjo 16×16 , a utilização já alcança 5,1% das LUTs e 7,1%

dos CLBs. No caso do arranjo 32×32 , observa-se um aumento mais expressivo, chegando a 19,1% das LUTs e 28,1% dos CLBs disponíveis no dispositivo.

A Figura 27(a) apresenta a quantidade absoluta de recursos utilizados para cada configuração do arranjo sistólico. Observa-se que o crescimento do consumo de recursos ocorre aproximadamente de forma quadrática com o aumento da dimensão do arranjo. Esse comportamento é esperado, pois um arranjo sistólico de dimensão $N \times N$ é composto por N^2 elementos de processamento (PEs), sendo que cada elemento contém lógica dedicada para realizar operações de multiplicação e acumulação.

A configuração de maior escala analisada corresponde ao arranjo 64×64 . Conforme apresentado na Tabela 5 e ilustrado nas Figuras 27(b) e 27(a), essa configuração utiliza aproximadamente 174 060 LUTs, 149 844 registradores e 27 521 CLBs. Em termos percentuais, isso corresponde a 76,1% das LUTs, 32,5% dos registradores e 96,1% dos CLBs disponíveis no dispositivo.

Figura 27 – LUTs, registradores e CLBs em função do tamanho do arranjo sistólico a 333 MHz.



((a)) Utilização absoluta de recursos.

((b)) Utilização percentual do FPGA.

Autoria: Própria.

Esses resultados indicam que, embora o aumento do tamanho do arranjo proporcione maior paralelismo computacional, ele também impõe uma demanda significativamente maior por recursos lógicos da FPGA. Dessa forma, observa-se que o arranjo 64×64 se aproxima do limite físico do dispositivo, evidenciando que a escalabilidade da arquitetura passa a ser limitada pela capacidade de recursos disponíveis na plataforma utilizada.

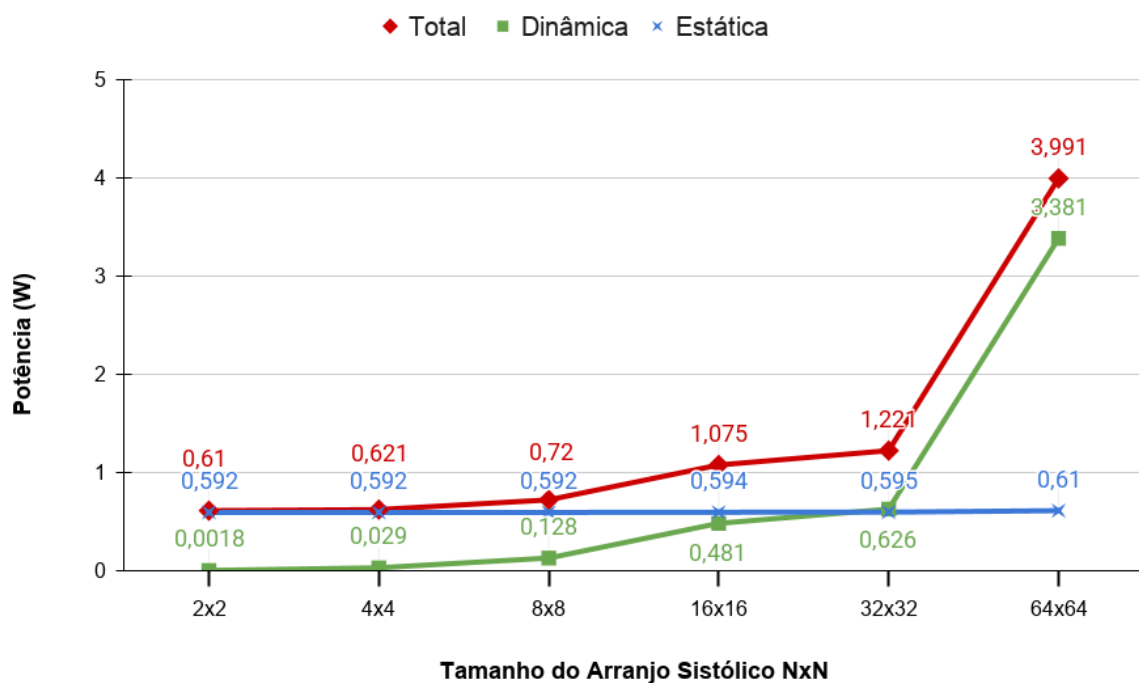
4.1.7 Potência Consumida

A análise de potência do acelerador foi realizada a partir da *netlist* implementada, utilizando informações de atividade de comutação obtidas por meio de arquivos SAIF (*Switching Activity Interchange Format*). Esse formato é amplamente empregado em fluxos

de projeto digital para representar a atividade dinâmica dos sinais internos do circuito ao longo da execução, permitindo uma estimativa de potência mais realista do que aquela baseada apenas em taxas de chaveamento médias assumidas pela ferramenta.

A Figura 28 apresenta a evolução da potência total, dinâmica e estática em função do tamanho do arranjo sistólico. Observa-se que, para as menores configurações, a potência total permanece próxima de um valor quase constante, pois a parcela estática do dispositivo domina o consumo global. Nas configurações de 2×2 e 4×4 , por exemplo, a potência dinâmica ainda é muito reduzida, o que faz com que a potência total fique em torno de aproximadamente 610 mW e 620 mW, respectivamente.

Figura 28 – Estimativa de Potência vs Tamanho do Arranjo Sistólico @333 MHz



Autoria: Própria.

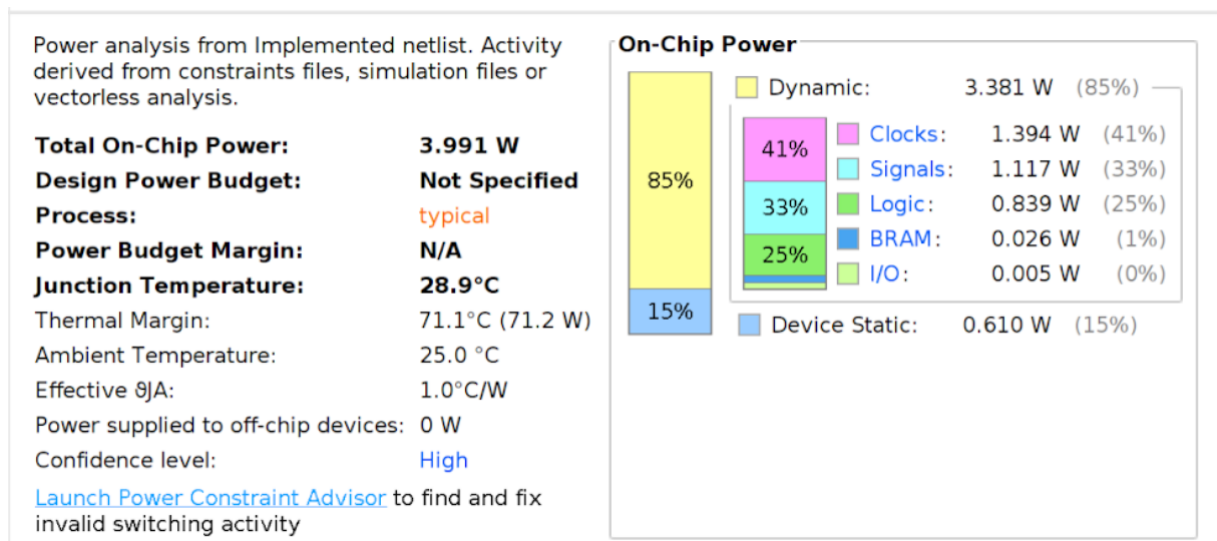
À medida que o tamanho do arranjo aumenta, a potência dinâmica cresce de forma progressiva, refletindo o aumento da quantidade de elementos de processamento ativos, registradores e sinais em comutação. Esse comportamento torna-se mais evidente a partir da configuração 8×8 , em que a potência dinâmica já alcança 128 mW, e segue aumentando para 481 mW em 16×16 e 626 mW em 32×32 . Nesses casos, embora a potência estática ainda permaneça praticamente constante, a contribuição da potência dinâmica passa a ter impacto significativo sobre a potência total.

O crescimento mais expressivo ocorre na configuração 64×64 , na qual a potência total atinge 3,991 W, sendo 3,381 W correspondentes à potência dinâmica e aproximada-

mente 610 mW à potência estática. Isso indica que, para arranjos maiores, o consumo do acelerador deixa de ser dominado pelas características intrínsecas do FPGA em repouso e passa a ser fortemente influenciado pela atividade de chaveamento gerada pela computação paralela do arranjo sistólico.

A decomposição interna da potência dinâmica ilustrada na figura 29, mostra ainda que a maior parcela do consumo está associada à rede de *clock* e aos sinais de interconexão. Para a configuração analisada de maior porte, cerca de 41% da potência dinâmica está relacionada aos *clocks*, 33% aos sinais, 25% à lógica combinacional e sequencial, enquanto BRAM e interfaces de entrada/saída apresentam contribuição pouco significativa. Esse resultado é coerente com a natureza da arquitetura proposta, uma vez que arranjos sistólicos grandes exigem intensa distribuição de relógio e ampla movimentação de dados entre elementos de processamento vizinhos.

Figura 29 – Decomposição de potencia dinâmica para o tamanho de arranjo sistólico de 64x64



Autoria: Própria.

Portanto, os resultados mostram que o aumento do paralelismo do acelerador traz como consequência um crescimento substancial da potência dinâmica. Esse comportamento é esperado e evidencia que, em arquiteturas sistólicas implementadas em FPGA, o escalonamento do desempenho deve ser analisado em conjunto com o custo energético, especialmente quando se utilizam arranjos de maior dimensão.

4.2 Comparação com Implementação em Software

4.2.1 Implementação em CPU

Como referência de desempenho, foi implementada uma versão da multiplicação de matrizes executada inteiramente em software no processador do sistema embarcado (ARM Cortex A53 @1.2 GHz). Essa implementação utiliza o algoritmo clássico de multiplicação de matrizes, baseado em três laços aninhados responsáveis pelo cálculo dos produtos e acumulações necessários para gerar cada elemento da matriz resultante.

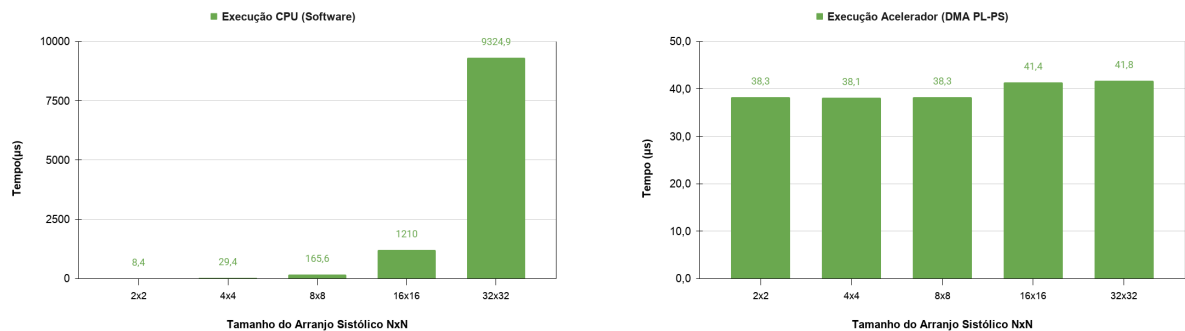
O tempo de execução da implementação em software foi medido por meio de temporização no processador e convertido para microssegundos. Conforme mostrado na Figura 30(a), o tempo de execução cresce rapidamente com o aumento da dimensão das matrizes, passando de $8,4 \mu s$ para a configuração 2×2 até $9314,3 \mu s$ para a configuração 32×32 .

Esse comportamento é coerente com a complexidade computacional do algoritmo convencional de multiplicação de matrizes, que é da ordem de $O(N^3)$. Assim, à medida que o tamanho do problema aumenta, o número de operações necessárias cresce de forma cúbica, elevando significativamente o tempo de processamento em software, como pode ser observado na Figura 30(a).

4.2.2 Comparação com o Acelerador em Hardware

A comparação entre o acelerador implementado em FPGA e a latência teórica do arranjo sistólico é apresentada na Figura 30(b). Nessa figura, observa-se que o tempo medido do acelerador FPGA, considerando a comunicação entre PS e PL via DMA, permanece relativamente baixo mesmo com o aumento da dimensão das matrizes, variando de $38,30 \mu s$ para 2×2 até $41,80 \mu s$ para 32×32 .

Figura 30 – Comparação entre execução em software e desempenho do acelerador FPGA.

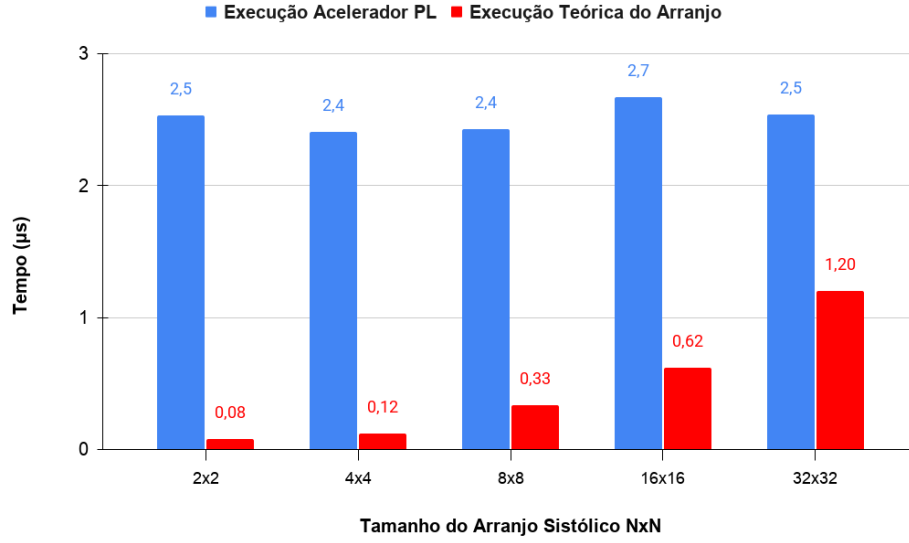


((a)) Tempo de execução da implementação em software.

((b)) Execução considerando todo o Sistema (PS+PL).

Autoria: Própria.

Figura 31 – Comparação entre o acelerador Ps e a execução teórica do arranjo sistólico.



Autoria: Própria.

Por sua vez, a execução teórica do arranjo sistólico, também apresentada na Figura 30(b), cresce de forma previsível com o tamanho do arranjo, partindo de $0,08 \mu s$ em 2×2 e atingindo $1,20 \mu s$ em 32×32 . A proximidade entre o valor medido e o valor teórico para matrizes maiores indica que, nessas configurações, o acelerador consegue aproveitar de forma mais eficiente o paralelismo espacial do arranjo sistólico.

Ainda assim, nota-se na Figura 30(b) que o tempo medido em hardware é sistematicamente superior à execução teórica. Essa diferença é explicada principalmente pela sobrecarga associada à transferência de dados entre memória e acelerador por meio do barramento AXI e do mecanismo DMA, além de custos adicionais de controle e sincronização entre o *Processing System* (PS) e o *Programmable Logic* (PL).

4.2.3 Análise de Speedup

A partir dos tempos de execução medidos em software e em hardware, foi calculado o *speedup* do acelerador, definido pela razão entre o tempo de execução da implementação em software e o tempo de execução da implementação em hardware, conforme a Equação 4.1.

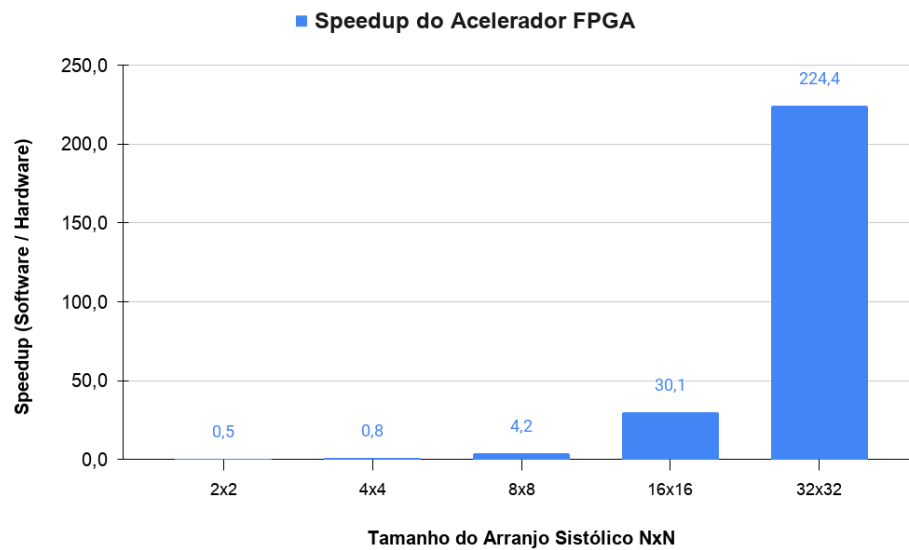
$$\text{Speedup} = \frac{T_{\text{software}}}{T_{\text{hardware}}} \quad (4.1)$$

Os resultados obtidos são apresentados na Figura 32. Observa-se que, para matrizes pequenas, o ganho de desempenho ainda é reduzido: aproximadamente $0,5\times$ para 2×2 e $0,8\times$ para 4×4 . Nesses casos, o custo de comunicação e acionamento do acelerador

representa uma parcela significativa do tempo total de execução, o que limita o benefício do processamento em hardware.

Entretanto, conforme mostrado na Figura 32, o ganho cresce rapidamente com o aumento da dimensão das matrizes. Para a configuração 8×8 , o *speedup* já atinge aproximadamente $4,2\times$. Para 16×16 , o ganho sobe para cerca de $30,1\times$, e para 32×32 alcança aproximadamente $215,4\times$.

Figura 32 – Speedup do acelerador FPGA em relação à implementação em software para diferentes tamanhos do arranjo sistólico.



Autoria: Própria.

Esses resultados evidenciam que o acelerador em FPGA se torna progressivamente mais vantajoso à medida que o tamanho do problema aumenta. Em outras palavras, quando a carga computacional passa a dominar sobre a sobrecarga de comunicação, o paralelismo intrínseco do arranjo sistólico torna-se mais efetivo, como evidenciado pelo comportamento da Figura 32.

De forma geral, a análise conjunta das Figuras 30(a), 30(b) e 32 demonstra que a arquitetura proposta apresenta elevado potencial para aceleração de operações matriciais, especialmente em problemas de maior dimensão.

5 Conclusões e Trabalhos Futuros

5.1 Síntese dos Resultados Obtidos

Este trabalho apresentou o desenvolvimento e avaliação experimental de um acelerador baseado em arranjo sistólico implementado em FPGA para operações de multiplicação de matrizes. A arquitetura proposta foi projetada de forma parametrizável, permitindo a variação do tamanho do arranjo sistólico $N \times N$ e possibilitando a análise de sua escalabilidade em termos de desempenho, utilização de recursos, consumo de potência e restrições de temporização.

Os resultados experimentais demonstraram que o aumento do tamanho do arranjo sistólico proporciona um crescimento significativo do paralelismo computacional. Conforme observado na análise de desempenho, o ganho de aceleração aumenta progressivamente com o tamanho do arranjo, atingindo um fator superior a 240, $4\times$ na configuração 32×32 . Esse resultado evidencia a eficiência do paralelismo espacial proporcionado pela arquitetura sistólica na aceleração de operações matriciais.

A análise de ciclos de clock também reforça essa tendência. Enquanto a implementação em software executada no processador apresenta crescimento significativo no número de ciclos para matrizes maiores, a solução baseada em hardware mantém um número de ciclos praticamente constante após determinado tamanho de arranjo, refletindo a natureza altamente paralela do processamento realizado no acelerador.

No entanto, o aumento do paralelismo também implica maior consumo de recursos da FPGA. Os resultados mostram que a utilização de LUTs, registradores e CLBs cresce aproximadamente de forma quadrática com o tamanho do arranjo. Para o caso do arranjo 64×64 , a utilização atingiu aproximadamente 76,1% das LUTs e 96,1% dos CLBs disponíveis no dispositivo, indicando que essa configuração se aproxima do limite de capacidade da FPGA utilizada.

Do ponto de vista de temporização, observou-se que o *slack* de *setup* diminui à medida que o arranjo aumenta de tamanho. Para arranjos menores, as margens de temporização são confortáveis, enquanto o arranjo 64×64 apresenta *slack* negativo, indicando dificuldades no fechamento de temporização para frequências elevadas. Esse comportamento é esperado, uma vez que o aumento do número de elementos de processamento eleva a complexidade do roteamento e o comprimento dos caminhos críticos.

Em relação ao consumo de potência, verificou-se que a potência dinâmica cresce significativamente com o aumento do tamanho do arranjo, alcançando aproximadamente

3,38 W na configuração 64×64 . Já a potência estática permanece praticamente constante, o que indica que o aumento do consumo energético está diretamente relacionado à atividade das unidades de processamento.

Esses resultados são consistentes com trabalhos da literatura que utilizam arquiteturas baseadas em arranjos sistólicos para acelerar aplicações de aprendizado profundo e processamento matricial. De acordo com (YANG et al., 2018), aceleradores baseados em arranjos sistólicos permitem explorar alto paralelismo e eficiência computacional, embora o aumento da dimensão do arranjo implique maior utilização de recursos de hardware e aumento do consumo de energia. Além disso, essas arquiteturas são particularmente adequadas para operações de multiplicação de matrizes e convoluções, amplamente utilizadas em algoritmos de aprendizado profundo.

De forma geral, os experimentos realizados demonstram que a arquitetura proposta é capaz de acelerar significativamente operações matriciais, evidenciando os principais compromissos de projeto entre desempenho, utilização de recursos e consumo de energia.

5.2 Limitações do Trabalho

Apesar dos resultados promissores obtidos, algumas limitações podem ser identificadas no presente trabalho.

Primeiramente, a implementação das unidades MAC foi realizada utilizando recursos lógicos gerais da FPGA, sem a utilização de blocos dedicados DSP. Essa decisão simplifica o projeto, porém limita o desempenho máximo e aumenta a utilização de LUTs.

Outra limitação refere-se às restrições de temporização observadas em arranjos sistólicos de maior dimensão. O aumento do número de elementos de processamento provoca crescimento no comprimento dos caminhos críticos, dificultando o fechamento de temporização em frequências elevadas.

Além disso, o fluxo de dados utilizado não incorpora técnicas avançadas de otimização de acesso à memória, como *tiling*, reutilização de dados em múltiplos níveis de memória ou técnicas de redução de tráfego entre memória e unidades de processamento.

Por fim, a avaliação experimental concentrou-se principalmente em métricas de síntese e implementação. Uma análise baseada em aplicações reais de aprendizado profundo poderia fornecer uma caracterização mais completa do desempenho do acelerador.

5.3 Possíveis Melhorias

Diversas melhorias podem ser exploradas para aumentar o desempenho e a eficiência da arquitetura proposta.

Uma melhoria importante consiste na utilização de blocos DSP da FPGA para implementação das unidades multiplicadoras-acumuladoras. Essa abordagem permitiria reduzir o uso de LUTs e aumentar a frequência máxima de operação.

Outra possibilidade é a introdução de técnicas de *pipelining* mais profundas, com o objetivo de reduzir o comprimento dos caminhos críticos e melhorar o fechamento de temporização em arranjos maiores.

Também podem ser exploradas técnicas de otimização do fluxo de dados, como reutilização de dados e organização hierárquica de memória, reduzindo o tráfego de dados entre memória e unidades de processamento.

Além disso, a introdução de mecanismos de controle mais sofisticados pode permitir o suporte a diferentes algoritmos de aprendizado de máquina e operações matriciais mais complexas.

5.4 Direções para Trabalhos Futuros

Como continuidade deste trabalho, diversas direções de pesquisa podem ser exploradas.

Uma possibilidade é a implementação de arranjos sistólicos maiores ou arquiteturas compostas por múltiplos arranjos interconectados, aumentando ainda mais o paralelismo computacional. Outra direção promissora é o desenvolvimento de estratégias automáticas de mapeamento de algoritmos de aprendizado profundo para o arranjo sistólico. Conforme discutido por (YANG et al., 2018), o desempenho dessas arquiteturas depende fortemente da forma como os dados são distribuídos e propagados no arranjo.

Também pode ser investigada a integração do acelerador a processadores baseados na arquitetura RISC-V por meio da extensão do conjunto de instruções (*Instruction Set Architecture – ISA*).

Nesse cenário, operações de multiplicação de matrizes poderiam ser expostas ao software como instruções especializadas, permitindo que o processador acione diretamente o acelerador em nível de instrução. Essa abordagem reduz a sobrecarga de comunicação associada ao uso de interfaces externas, como DMA, e possibilita uma integração mais eficiente entre o núcleo de processamento e o acelerador de hardware.

Uma implementação aberta do acelerador sistólico desenvolvido neste trabalho está disponível no repositório *SystolicCore*, que reúne o código-fonte do acelerador, scripts de simulação e documentação técnica do projeto, permitindo sua reprodução e extensão por outros pesquisadores (SILVA, 2026).

Como trabalho futuro, pode-se avaliar o acelerador em aplicações mais complexas,

como redes neurais convolucionais ou outros algoritmos intensivos em multiplicação de matrizes, permitindo analisar o desempenho da arquitetura em cenários reais de computação de alto desempenho.

Referências

CHEN, Z. *Hardware Accelerator of Matrix Multiplication on FPGAs*. Dissertação (Mestrado) — Uppsala University, 2018.

KUNG, H. T.; LEISERSON, C. E. Systolic arrays (for vlsi). In: *Sparse Matrix Proceedings 1978*. [S.l.]: Society for Industrial and Applied Mathematics, 1979. (SIAM), p. 256–282.

SILVA, V. F. *SystolicCore: Hardware Implementation of a Systolic Matrix Multiplication Accelerator*. 2026. <<https://github.com/valmyr/SystolicCore>>. Accessed: 2026.

SMEDING, T. *Fast Large-Integer Matrix Multiplication*. Dissertação (Mestrado) — Leiden University, 2018.

SYSTOLIC Arrays - Computer Science Topic. ScienceDirect, 2023. Disponível em: <<https://www.sciencedirect.com/topics/computer-science/systolic-arrays>>.

XILINX, INC. *ZCU106 Evaluation Board User Guide*. [S.l.], 2019.

YANG, Z. et al. Systolic array based accelerator and algorithm mapping for deep learning algorithms. In: SPRINGER. *Proceedings of the International Conference on Artificial Intelligence and Computational Intelligence*. [S.l.], 2018.